

用 Trae 从零搭建 ERP 系统（完整版）

零基础 18 章实战指南 · 基于 RCA6.0 汽配外贸 ERP 系统 · 含全部 Prompt 与代码

前言

本教程基于一个真实运行的 ERP 系统（RCA6.0 汽配外贸 ERP），手把手教你用字节跳动出品的免费 AI 原生 IDE **Trae** 从零开始搭建一个完整的企业级管理系统。教程覆盖 14 个核心业务模块，每个模块都给出可直接复制的 Trae Builder Prompt、完整的代码示例、详细的操作步骤和避坑指南。

适合人群：从未写过代码的业务人员、略懂前端的初学者、希望用 AI 提效的开发者。

学习时长：理论 8-12 小时 + 实操 20-30 小时，可完成一个可用的 ERP MVP。

学习成果：能用 Trae 独立搭建一个包含客户、产品、订单、库存、财务等模块的全栈 ERP 系统，并部署上线。

第一章 Trae 简介与环境准备

1.1 Trae 是什么

Trae（中文名：迹）是字节跳动在 2025 年初推出的一款 AI 原生集成开发环境（AI-Native IDE）。所谓"AI 原生"，指的是 Trae 从设计之初就把 AI 当作核心功能而非附加插件——它的整个交互逻辑都围绕"与 AI 对话来编写代码"展开。你可以把它理解为"内置了 ChatGPT/Claude 能力的 VS Code"。

Trae 基于开源的 VS Code 内核构建，因此所有 VS Code 的插件、快捷键、主题都能在 Trae 中直接使用。如果你之前用过 VS Code，迁移到 Trae 几乎零成本。如果你从未用过任何 IDE，也不用担心——Trae 的界面设计简洁直观，配合 AI 对话能力，新手反而更容易上手。

Trae 与同类产品（如 Cursor）相比最大的优势是：**完全免费、国内可直接访问、原生支持中文**。这意味着你不需要付费订阅、不需要科学上网、不需要英文对话，就能享受到与 Cursor 接近的 AI 编程体验。对国内开发者，尤其是零基础学习者来说，这是目前最友好的选择。

1.2 为什么选 Trae 而不是其他工具

维度	Trae	Cursor	VS Code+Copilot	智谱清言
价格	免费	\$20/月	\$10/月	免费
国内访问	直接	需代理	不稳定	直接
AI 对话	内置	内置	插件	网页
多文件生成	支持	支持	不支持	不支持

维度	Trae	Cursor	VS Code+Copilot	智谱清言
代码补全	支持	支持	支持	不支持
中文界面	原生	需配置	需配置	原生
学习曲线	低	中	中	极低
适合人群	国内开发者	专业开发者	VS Code 老用户	纯小白

从对比表可以看出，Trae 在"免费 + 国内可用 + AI 原生"三个维度上做到了最佳平衡。对于本教程的目标读者——零基础学习者——Trae 是不二之选。

1.3 下载与安装

1.3.1 Windows 安装

步骤 1：访问 Trae 官网

打开浏览器，访问 <https://www.trae.cn/>。这是 Trae 的官方首页，页面正中间会有一个明显的"下载 Trae"按钮。

步骤 2：选择 Windows 版本

点击下载按钮后，会弹出系统选择框。选择"Windows"系统，会自动下载 .exe 安装包（约 100MB）。

步骤 3：运行安装包

下载完成后，双击 .exe 文件运行。Windows 可能弹出"用户账户控制"提示，点击"是"继续。

步骤 4：选择安装路径

默认安装路径是 `C:\Users\{用户名}\AppData\Local\Trae`。如果你想安装到其他位置，点击"浏览"选择。建议保持默认，避免后续路径问题。

步骤 5：等待安装完成

点击"安装"按钮，等待 1-3 分钟。安装完成后会自动创建桌面快捷方式。点击"完成"退出安装程序。

1.3.2 macOS 安装

步骤 1：访问官网下载

同样访问 <https://www.trae.cn/>，选择"macOS"系统下载 .dmg 文件。注意区分 Apple Silicon (M1/M2/M3 芯片) 和 Intel 芯片版本。查看芯片方法：点击左上角苹果图标 → 关于本机 → 查看"芯片"字段。

步骤 2：挂载 dmg 镜像

下载完成后双击 .dmg 文件，会挂载一个磁盘镜像，里面显示 Trae.app 图标和 Applications 文件夹快捷方式。

步骤 3：拖动到 Applications

把 Trae.app 拖动到右侧的 Applications 文件夹，等待复制完成（约 30 秒）。

步骤 4：首次启动

在启动台或Applications文件夹中找到Trae，双击启动。首次启动会提示"是否打开从互联网下载的应用"，点击"打开"。

1.3.3 Linux 安装

Linux 用户访问官网下载 .AppImage 或 .deb 包。 .AppImage 是免安装版本，下载后执行 `chmod +x` 加可执行权限即可运行； .deb 包用 `sudo dpkg -i` 命令安装。

1.4 首次启动配置

步骤 1：登录账号

首次启动 Trae 会显示欢迎页面，点击"登录"。可以使用字节跳动账号、手机号或邮箱注册登录。登录后可同步配置到云端。

步骤 2：选择界面语言

Trae 默认检测系统语言。如果你的界面是英文，按 `Ctrl+Shift+P` 打开命令面板，输入"Configure Display Language"，选择"中文（简体）"，重启 Trae 生效。

步骤 3：选择主题

在欢迎页面选择主题。推荐选择"深色主题"（Dark+），编程时眼睛更舒服。也可以后续在 文件 → 首选项 → 主题 中修改。

步骤 4：信任文件夹

首次打开项目文件夹时，Trae 会提示"是否信任此文件夹中的文件作者"。勾选"信任"并点击"是"。这是 VS Code 系的安全机制。

1.5 必装插件

虽然 Trae 自带了很多功能，但开发 Next.js + Tailwind 项目还需要装几个插件来提升体验。安装方法：点击左侧活动栏的"扩展"图标（或按 `Ctrl+Shift+X`），搜索插件名点击安装。

插件名	用途	必装
Tailwind CSS IntelliSense	Tailwind 类名智能提示与预览	是
PostgreSQL	在 IDE 内连接 Supabase 数据库	是
ESLint	JavaScript/TypeScript 代码检查	是
Prettier	代码格式化	是
GitLens	Git 历史增强	推荐
Auto Rename Tag	HTML/JSX 标签自动重命名	推荐
Path Intellisense	文件路径自动补全	推荐
Material Icon Theme	文件图标美化	可选

1.6 Trae 三大核心功能

Trae 的 AI 能力主要通过三个入口使用，掌握这三个功能你就掌握了 Trae 80% 的能力。

1.6.1 Chat 对话（Ctrl+L）

按 **Ctrl+L**（Mac 是 **Cmd+L**）唤起右侧 Chat 面板。这是与 AI 单轮对话的入口，适合：

- **提问**：解释某段代码、查询 API 用法、问技术概念
- **生成片段**：让 AI 写一个函数、一段配置、一条 SQL
- **调试**：粘贴报错信息让 AI 解释原因并给出修复方案
- **重构建议**：选中代码后让 AI 给出优化建议

Chat

模式下

AI

只会给出代码建议，不会直接修改你的文件。你需要手动复制粘贴。适合"我只想问一下"的场景。

1.6.2 Builder 多文件生成（Ctrl+I）

按 **Ctrl+I**（Mac 是 **Cmd+I**）唤起 **Builder** 内联输入框。这是 **Trae** 最强大的功能——它能**直接修改多个文件**。适合：

- **创建完整功能**：例如"创建一个客户管理页面，含列表、表单、API"，AI 会一次性创建 3-5 个文件
- **批量重构**：例如"把所有 console.log 替换为 logger.info"
- **添加新模块**：例如"添加一个供应商管理模块，参照客户管理的结构"
- **修复跨文件 bug**：AI 会自动找到相关文件并修改

Builder 模式下 AI 会显示一个 diff 视图，让你 review 后再决定是否接受修改。这是 Trae 区别于普通 Chat 工具的核心价值。

1.6.3 Tab 代码补全

在编辑器中输入代码时，Trae 会自动弹出灰色补全建议。按 **Tab** 键接受。与传统 IDE 的补全不同，Trae 的补全是**基于上下文理解的**——它不仅补全当前行的关键字，还能根据你正在写的功能预测接下来的多行代码。

例如，你刚写完一个函数定义 `function createCustomer(name: string)`，Trae 可能直接补全整个函数体（包括参数校验、数据库写入、返回结果），按 **Tab** 一键接受。这比手动写快 3-5 倍。

1.7 学习本教程的前置准备

在开始第二章之前，请确认你已经完成：

- ☒ Trae 已安装并能正常启动
- ☒ 已登录 Trae 账号
- ☒ 界面已切换为中文
- ☒ 已安装必装插件（Tailwind CSS IntelliSense、ESLint、Prettier）
- ☒ 电脑已连接网络（开发过程需要访问 Supabase 云服务）
- ☒ 准备好 4-6 小时的专注学习时间（每天 1-2 小时，3-5 天完成全部教程）

第二章 项目初始化与基础架构

2.1 安装 Node.js 与 pnpm

2.1.1 安装 Node.js

Node.js 是 JavaScript 的运行环境，所有前端项目都依赖它。我们安装 LTS（长期支持）版本，目前是 20.x。

步骤 1：访问 Node.js 官网

打开浏览器访问 <https://nodejs.org/>。页面会显示两个版本按钮，点击"LTS"版本下载（Windows 是 .msi 文件，Mac 是 .pkg 文件）。

步骤 2：运行安装

双击安装包，全程点击"下一步"。注意勾选"Add to PATH"选项（默认已勾选），这样可以在命令行直接使用 node 命令。

步骤 3：验证安装

在 Trae 中按 `Ctrl+\`` 打开终端（Terminal），输入：

```
node --version
```

如果显示 `v20.x.x` 表示安装成功。同时输入 `npm --version` 应显示 `10.x.x`。

2.1.2 安装 pnpm

pnpm 是比 npm 更快、更省磁盘的包管理器。本教程推荐使用 pnpm。在 Trae 终端执行：

```
npm install -g pnpm
pnpm --version
```

应显示 `9.x.x`。

2.2 创建 Next.js 14 项目

步骤 1：在 Trae 中打开文件夹

在电脑上创建一个空文件夹，例如 `D:\projects\rca-erp`。在 Trae 中点击 文件 → 打开文件夹，选择这个文件夹。

步骤 2：在 Trae 终端执行创建命令

按 `Ctrl+\`` 打开终端，输入以下命令（注意末尾的点表示在当前目录创建）：

```
npx create-next-app@latest . --typescript --tailwind --eslint --app --src-dir --import-alias "@/*" --use-pnpm
```

命令参数说明：

- `--typescript` 使用 TypeScript（必须）
- `--tailwind` 集成 Tailwind CSS（必须）
- `--eslint` 启用 ESLint 代码检查
- `--app` 使用 App Router（Next.js 13+ 的新路由）

- `--src-dir` 把源代码放在 `src/` 目录（推荐）
- `--import-alias` 设置路径别名，可以用 `@/` 代替 `src/`
- `--use-pnpm` 使用 `pnpm` 作为包管理器

步骤 3：回答交互提示

如果提示"是否使用 TypeScript?"按 `Y` 确认。如果提示"目录不为空，是否继续?"也按 `Y`。整个创建过程约 1-2 分钟。

步骤 4：验证项目创建成功

在终端输入 `pnpm dev` 启动开发服务器。打开浏览器访问 `http://localhost:3000`，看到 Next.js 默认欢迎页面表示成功。按 `Ctrl+C` 停止服务器。

2.3 项目结构初识

创建完成后，Trae 左侧资源管理器会显示项目结构：

路径	说明
<code>src/app/</code>	App Router 主目录，所有页面放这里
<code>src/app/layout.tsx</code>	根布局文件，所有页面共享的外壳
<code>src/app/page.tsx</code>	首页，对应路由 <code>/</code>
<code>src/app/globals.css</code>	全局样式，Tailwind 在这里引入
<code>src/components/</code>	自定义组件目录（需手动创建）
<code>src/lib/</code>	工具函数和库目录（需手动创建）
<code>public/</code>	静态资源目录（图片、字体等）
<code>package.json</code>	项目依赖清单
<code>tsconfig.json</code>	TypeScript 配置
<code>tailwind.config.ts</code>	Tailwind 配置
<code>next.config.mjs</code>	Next.js 配置
<code>.env.local</code>	环境变量文件（需手动创建，存敏感信息）

现在创建后续需要的目录。在 Trae 终端执行：

```
mkdir -p src/components/ui src/components/layout src/components/common
mkdir -p src/lib/supabase src/hooks src/types
mkdir -p src/app/api src/app/(dashboard)
mkdir -p sql
```

2.4 安装依赖

接下来安装 ERP 项目需要的所有第三方依赖。在 Trae 终端逐条执行：

```
# 1. 安装 Supabase 客户端（数据库）
pnpm add @supabase/supabase-js @supabase/ssr

# 2. 安装 shadcn/ui 依赖（UI 组件库）
pnpm add class-variance-authority clsx tailwind-merge tailwindcss-animate
pnpm add @radix-ui/react-dialog @radix-ui/react-dropdown-menu
pnpm add @radix-ui/react-label @radix-ui/react-select @radix-ui/react-slot
pnpm add @radix-ui/react-toast @radix-ui/react-checkbox @radix-ui/react-popover

# 3. 安装表单和数据请求
pnpm add react-hook-form @hookform/resolvers zod
pnpm add @tanstack/react-query

# 4. 安装图表和工具
pnpm add recharts date-fns lucide-react

# 5. 安装 Excel/PDF 处理
pnpm add xlsx jspdf jspdf-autotable

# 6. 安装开发依赖
pnpm add -D @types/node prettier
```

2.5 shadcn/ui 初始化与组件添加

shadcn/ui 不是传统的 npm 组件库，而是一套“复制即用”的组件代码集合。它会把组件源码直接复制到你的项目中，你可以自由修改。

步骤 1：初始化 shadcn/ui

在 Trae 终端执行：

```
npx shadcn-ui@latest init
```

提示选择样式时选"Default"，选择 base color 时选"Slate"。

步骤 2：添加常用组件

执行以下命令一次性添加本教程需要的所有组件：

```
npx shadcn-ui@latest add button input label select dialog
npx shadcn-ui@latest add table dropdown-menu toast checkbox
npx shadcn-ui@latest add badge separator tabs popover card
npx shadcn-ui@latest add avatar skeleton sonner
```

步骤 3：验证组件

完成后 `src/components/ui/` 目录下应该有十几个 .tsx 文件，每个对应一个组件。这些都是可修改的源码。

2.6 环境变量配置

步骤 1：用 Trae Builder 创建 .env.local 文件

按 `Ctrl+I` 唤起 Builder，输入以下 Prompt：

在项目根目录创建 `.env.local` 文件，包含以下内容：

```
NEXT_PUBLIC_SUPABASE_URL=your-supabase-project-url
NEXT_PUBLIC_SUPABASE_ANON_KEY=your-supabase-anon-key
SUPABASE_SERVICE_ROLE_KEY=your-supabase-service-role-key
```

```
NEXT_PUBLIC_APP_NAME=RCA6.0 ERP
NEXT_PUBLIC_APP_URL=http://localhost:3000
```

同时在项目根目录创建 `.env.example` 文件，内容相同但值替换为占位符，作为给其他开发者的模板。
同时确认 `.gitignore` 文件中包含 `.env.local` 一行，如果没有则添加。

步骤 2：验证文件创建

打开 `.env.local` 文件，应该能看到上述内容。实际的 Supabase URL 和 API Key 需要在第三章创建 Supabase 项目后获取，暂时保留占位符即可。

2.7 启动开发服务器验证

最后验证整个项目能正常运行。在 Trae 终端执行 `pnpm dev`，然后打开浏览器访问 `http://localhost:3000`。如果你看到 Next.js 默认的欢迎页面（黑色背景，Next.js logo），说明项目初始化成功。

完成检查清单：

- ☒ Next.js 14 项目能正常启动
- ☒ `src/` 下有 `app/components/lib/hooks/types` 目录
- ☒ `package.json` 包含约 20 个依赖
- ☒ `src/components/ui/` 下有十几个 shadcn 组件
- ☒ 根目录有 `.env.local` 和 `.env.example` 文件
- ☒ `.gitignore` 包含 `.env.local`

第三章 数据库设计与 Supabase 配置

3.1 注册 Supabase 账号与创建项目

Supabase 是一个开源的 Firebase 替代品，提供 PostgreSQL 数据库、认证、存储、实时订阅等服务。它的免费版包含 500MB 数据库 + 50000 月活用户 + 1GB 存储，对小型 ERP 完全够用。

步骤 1：访问 Supabase 官网

打开浏览器访问 <https://supabase.com/>。点击右上角"Start your project"注册账号。可以用 GitHub 账号一键登录（推荐），无需另注册。

步骤 2：创建新项目

登录后进入 Dashboard，点击"New project"。填写项目名称 `rca-erp`，设置数据库密码（建议用密码管理器生成强密码并保存好，这个密码无法找回），选择区域"Southeast Asia (Singapore)"（离中国最近的区域，延迟最低）。

步骤 3：等待项目初始化

点击"Create new project", Supabase 会用约 2 分钟初始化数据库和 API。完成后自动跳转到项目 Dashboard。

步骤 4：获取连接信息

在 Dashboard 左侧点击"Project Settings"（齿轮图标）→ "API"。页面会显示：

- Project URL（如 <https://xxxxx.supabase.co>）
- anon public key（公开密钥，前端用）
- service_role key（服务端密钥，保密！）

把这三个值复制到项目的 .env.local 文件中对应位置。

3.2 在 Trae 中创建 Supabase 客户端

现在让 Trae 帮我们创建 Supabase 客户端文件。我们需要两个版本：一个用于浏览器端（用 anon key，受 RLS 限制），一个用于服务端（用 service_role key，绕过 RLS）。

按 **Ctrl+I** 唤起 Builder，输入以下 Prompt：

在 src/lib/supabase/ 目录下创建三个文件：

1. client.ts - 浏览器端 Supabase 客户端
 - 使用 createBrowserClient from @supabase/ssr
 - 读取环境变量 NEXT_PUBLIC_SUPABASE_URL 和 NEXT_PUBLIC_SUPABASE_ANON_KEY
 - 导出 supabase 单例
 - 完整代码示例：

```
import { createBrowserClient } from '@supabase/ssr';

export const supabase = createBrowserClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL!,
  process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
);
```

2. server.ts - 服务端 Supabase 客户端（用于 Next.js Server Components 和 API Routes）
 - 使用 createServerClient from @supabase/ssr
 - 读取 cookies() 处理用户会话
 - 导出 createClient() 函数（每次调用返回新实例）
 - 完整代码示例：

```
import { createServerClient } from '@supabase/ssr';
import { cookies } from 'next/headers';

export function createClient() {
  const cookieStore = cookies();

  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
```

```
process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
{
cookies: {
getAll() { return cookieStore.getAll(); },
setAll(cookiesToSet) {
cookiesToSet.forEach(({ name, value, options }) => cookieStore.set(name, value, options));
},
},
});
}
```

3. admin.ts - 管理员客户端（绕过 RLS，仅在 API Routes 中用）

- 使用 createClient from @supabase/supabase-js
- 使用 SUPABASE_SERVICE_ROLE_KEY（仅服务端可用）
- 用于需要绕过 RLS 的管理操作（如初始化数据）
- 完整代码示例：

```
import { createClient } from '@supabase/supabase-js';

export function createAdminClient() {
return createClient(
process.env.NEXT_PUBLIC_SUPABASE_URL!,
process.env.SUPABASE_SERVICE_ROLE_KEY!,
{ auth: { persistSession: false } }
);
}
```

请同时创建 src/lib/supabase/types.ts 文件，导出一个 Database 类型（暂时为 any，后续会自动生成）：

```
export type Database = any;
```

Trae 会自动创建这三个文件并在 src/lib/supabase/ 下展示。你可以点击每个文件查看内容。如果生成的代码有问题（比如导入了不存在的包），把报错信息复制给 Chat（Ctrl+L）让它修复。

3.3 设计数据库表结构

现在开始设计数据库表结构。基于 14 个模块，我们需要约 29 张表。让 Trae 帮我们生成完整的建表 SQL。

在 Trae 中创建 `sql/001_init.sql` 文件（在 Trae 资源管理器右键 `sql` 目录 → 新建文件），然后按 `Ctrl+I` 唤起 Builder，输入：

请帮我生成 PostgreSQL 建表 SQL，写入 `sql/001_init.sql` 文件。这是汽配外贸 ERP 系统，需要以下表：

【设计原则】

1. 所有表含 `id(uuid PK default gen_random_uuid())`, `code(text unique)`, `created_at`, `updated_at`, `deleted_at`(软删除)
2. 枚举用 `text + CHECK` 约束, 不用 PostgreSQL `enum` 类型
3. 金额用 `numeric(18,2)`, 数量用 `numeric(10,2)`
4. 外键建索引
5. 业务编码格式：前缀+日期+序号

【表清单】

1. users - 用户 (`email`, `password_hash`, `name`, `role[admin|sales|purchaser|warehouse|finance]`, `status`)
2. audit_logs - 操作日志 (`user_id`, `module`, `action`, `record_id`, `before_data jsonb`, `after_data jsonb`)
3. system_settings - 系统配置 (`key`, `value jsonb`)
4. exchange_rates - 汇率 (`currency`, `rate_to_cny`, `effective_date`)
5. customers - 客户 (`code`, `name`, `name_en`, `contact_person`, `country`, `level`, `status`, `credit_limit`, `payment_terms`, `preferred_currency`, `assigned_to`)
6. suppliers - 供应商 (`code`, `name`, `contact_person`, `country`, `level`, `main_category`, `lead_time_days`, `quality_rating`)
7. product_categories - 产品分类 (`code`, `name`, `parent_id` 自关联)
8. products - 产品 (`code`, `name`, `name_en`, `oem_number`, `category_id`, `brand`, `image_urls text[]`, `cost_price`, `sale_price`, `unit`, `weight_kg`, `status`, `applicable_models text[]`)
9. product_suppliers - 产品-供应商关联 (`product_id`, `supplier_id`, `is_primary`, `supply_price`)
10. inquiries - 询盘 (`code`, `customer_id`, `subject`, `source[email|phone|exhibition|website|wechat]`, `priority[high|medium|low]`, `status[pending|processing|quoted|closed|cancelled]`)
11. inquiry_items - 询盘明细 (`inquiry_id`, `product_id`, `quantity`, `target_price`)
12. quotations - 报价 (`code`, `customer_id`, `inquiry_id`, `pricing_status[pending|priced]`, `status[draft|sent|accepted|rejected|expired]`, `total_amount`, `currency`, `valid_until`)
13. quotation_items - 报价明细 (`quotation_id`, `product_id`, `quantity`, `unit_price`, `total_price generated`)
14. orders - 订单 (`code`, `customer_id`, `quotation_id`, `order_date`, `delivery_date`, `currency`, `total_amount`, `status[pending|confirmed|producing|shipped|completed|cancelled]`)
15. order_items - 订单明细 (`order_id`, `product_id`, `quantity`, `shipped_quantity`, `unit_price`)
16. purchase_requests - 请购单 (`code`, `order_id`, `status[pending|approved|rejected|converted]`)
17. purchase_request_items - 请购明细
18. purchase_orders - 采购订单 (`code`, `supplier_id`, `status[pending|approved|sent|partial_arrived|arrived|warehoused|paid]`)
19. purchase_order_items - 采购明细 (`quantity`, `arrived_quantity`, `warehoused_quantity`)
20. warehouses - 仓库 (`code`, `name`, `address`)
21. inventory - 库存 (`product_id`, `warehouse_id`, `quantity`, `frozen_quantity`, `available_quantity generated`)
22. stock_movements - 出入库流水 (`movement_type`, `product_id`, `quantity`, `reference_type`, `reference_id`)
23. shipments - 发货单 (`code`, `order_id`, `shipment_date`, `shipping_method[sea|air|express|land]`, `tracking_number`, `container_number`, `bl_number`)
24. shipment_items - 发货明细 (`shipment_id`, `order_item_id`, `quantity`, `cartons`, `weight_kg`)
25. receivables - 应收 (`code`, `customer_id`, `order_id`, `amount`, `received_amount`, `currency`, `due_date`, `status[pending|partial|received|overdue]`)
26. receivable_payments - 收款记录 (`receivable_id`, `amount`, `payment_date`)
27. payables - 应付 (类似 `receivables`)
28. documents - 单证 (`code`, `order_id`, `shipment_id`, `document_type[commercial_invoice|packing_list|bill_of_lading|...]`, `file_url`)

29. notifications - 通知 (`user_id`, `type`, `title`, `content`, `is_read`)

【输出要求】

- 完整 `CREATE TABLE` 语句，含主键、外键、索引、`CHECK` 约束
- 每张表前加注释说明用途
- 按"基础表→业务表→关联表"顺序排列
- 不要省略任何字段，每个字段都要写完整定义

Trae 会自动生成完整的 SQL 文件（约 300-500 行）。打开 `sql/001_init.sql` 检查内容是否符合预期。如果某些表字段不够完整，可以再让 Trae 补充。

3.4 在 Supabase SQL Editor 执行建表

步骤 1: 打开 SQL Editor

在 Supabase Dashboard 左侧菜单点击"SQL Editor", 点击"New query"新建查询。

步骤 2: 粘贴 SQL

回到 Trae, 打开 `sql/001_init.sql` 文件, `Ctrl+A` 全选 `Ctrl+C` 复制。回到 Supabase SQL Editor, `Ctrl+V` 粘贴到查询框。

步骤 3: 执行建表

点击右下角"Run"按钮（或按 `Ctrl+Enter`）。底部输出区会显示"Success. No rows returned."。如果有报错（红色文字），把错误信息复制给 Trae Chat 让它修复 SQL。

步骤 4: 验证表已创建

在 Supabase 左侧菜单点击"Table Editor", 应该能看到 29 张表都列出来了。点击任意表（如 customers）查看字段。

3.5 配置 RLS（行级安全）策略

Supabase 默认开启 RLS（Row Level Security），如果不配置策略，前端将无法读写任何数据。对于 MVP 阶段，我们暂时关闭 RLS 简化开发，后续上线前再开启。

在 Supabase SQL Editor 中执行以下 SQL（同样保存到 `sql/002_disable_rls.sql` 文件中）：

```
-- 临时关闭所有表的 RLS（开发阶段）
-- 注意：上线前必须重新启用并配置细粒度策略

ALTER TABLE users DISABLE ROW LEVEL SECURITY;
ALTER TABLE audit_logs DISABLE ROW LEVEL SECURITY;
ALTER TABLE system_settings DISABLE ROW LEVEL SECURITY;
ALTER TABLE exchange_rates DISABLE ROW LEVEL SECURITY;
ALTER TABLE customers DISABLE ROW LEVEL SECURITY;
ALTER TABLE suppliers DISABLE ROW LEVEL SECURITY;
ALTER TABLE product_categories DISABLE ROW LEVEL SECURITY;
ALTER TABLE products DISABLE ROW LEVEL SECURITY;
ALTER TABLE product_suppliers DISABLE ROW LEVEL SECURITY;
ALTER TABLE inquiries DISABLE ROW LEVEL SECURITY;
ALTER TABLE inquiry_items DISABLE ROW LEVEL SECURITY;
ALTER TABLE quotations DISABLE ROW LEVEL SECURITY;
ALTER TABLE quotation_items DISABLE ROW LEVEL SECURITY;
ALTER TABLE orders DISABLE ROW LEVEL SECURITY;
ALTER TABLE order_items DISABLE ROW LEVEL SECURITY;
ALTER TABLE purchase_requests DISABLE ROW LEVEL SECURITY;
ALTER TABLE purchase_request_items DISABLE ROW LEVEL SECURITY;
ALTER TABLE purchase_orders DISABLE ROW LEVEL SECURITY;
ALTER TABLE purchase_order_items DISABLE ROW LEVEL SECURITY;
ALTER TABLE warehouses DISABLE ROW LEVEL SECURITY;
ALTER TABLE inventory DISABLE ROW LEVEL SECURITY;
ALTER TABLE stock_movements DISABLE ROW LEVEL SECURITY;
ALTER TABLE shipments DISABLE ROW LEVEL SECURITY;
ALTER TABLE shipment_items DISABLE ROW LEVEL SECURITY;
ALTER TABLE receivables DISABLE ROW LEVEL SECURITY;
ALTER TABLE receivable_payments DISABLE ROW LEVEL SECURITY;
ALTER TABLE payables DISABLE ROW LEVEL SECURITY;
ALTER TABLE documents DISABLE ROW LEVEL SECURITY;
ALTER TABLE notifications DISABLE ROW LEVEL SECURITY;
```

安全提示：关闭 RLS 意味着任何拿到 anon key 的人都能读写所有数据。这在开发阶段可接受，但上线生产环境前必须重新启用 RLS 并配置细粒度策略。

3.6 生成 TypeScript 类型

Supabase 提供了 CLI 工具，能根据数据库表结构自动生成 TypeScript 类型。这样在代码中访问数据库就有完整的类型提示。

步骤 1：安装 Supabase CLI

在 Trae 终端执行：

```
npm install -g supabase
```

步骤 2：登录 Supabase

执行：

```
supabase login
```

会打开浏览器让你登录 Supabase 账号并授权 CLI。

步骤 3：生成类型

执行以下命令（把 YOUR_PROJECT_REF 替换为你的项目 ID，在 Supabase Dashboard → Settings → General 中可找到）：

```
supabase gen types typescript --project-id YOUR_PROJECT_REF --schema public > src/types/database.ts
```

步骤 4：验证类型文件

打开

`src/types/database.ts`，应该能看到所有表的类型定义。文件可能上千行，这是正常的。后续在代码中用 `supabase.from("customers").select()` 时会有完整的字段类型提示。

3.7 种子数据准备

空数据库没法测试功能，我们需要准备一些演示数据。让 Trae 生成种子数据 SQL：

按 `Ctrl+I` 唤起 Builder，输入：

在 `sql/003_seed_data.sql` 文件中生成演示数据 SQL，要求：

1. 5 个用户：1 `admin(admin@rca-erp.com)`，1 `sales(sales@rca-erp.com)`，1 `purchaser`，1 `warehouse`，1 `finance`。密码统一用 bcrypt 哈希：\$2a\$12\$xxx ("admin123"的哈希)
2. 1 个默认仓库：code=WH001，name=主仓库
3. 系统设置：company_name=RCA汽配贸易有限公司，default_currency=USD，default_trade_terms=FOB，default_payment_terms=T/T
4. 3 个汇率：USD/CNY=7.25，EUR/CNY=7.85，GBP/CNY=9.15
5. 5 个产品分类：制动系统，发动机配件，密封件，轴承，电气系统
6. 10 个产品：覆盖不同分类和品牌（如前制动片、后制动片、机油滤芯、气门室垫片、深沟球轴承等），每个有 `cost_price` 和 `sale_price`
7. 5 个客户：覆盖不同国家（中国、美国、德国、日本、阿联酋）和等级
8. 3 个供应商：国内不同地区的零件厂
9. 5 个询盘：不同状态和优先级
10. 3 个报价单：含明细
11. 3 个订单：1 个 `pending`，1 个 `confirmed`，1 个 `completed`
12. 库存初始化：每个产品在主仓库有 100-500 件库存

【要求】

- 用 `INSERT INTO` 语句，每个语句包含 `ON CONFLICT DO NOTHING` 防止重复
- 编码用固定值便于测试（如 CU001，PD001，IN-2026-0001）
- 时间用 `NOW() - INTERVAL 'X days'` 形式表达相对时间
- 数据之间外键关系要正确（如订单的 `customer_id` 必须存在）

生成后把 `sql/003_seed_data.sql` 的内容粘贴到 Supabase SQL Editor 执行。完成后用 Table Editor 检查 `customers` 表，应该能看到 5 条客户记录。

完成检查清单：

- ☒ Supabase 项目已创建并拿到 URL/Key
- ☒ `.env.local` 已填入真实凭证
- ☒ `src/lib/supabase/` 下有 `client.ts`, `server.ts`, `admin.ts`
- ☒ `sql/001_init.sql` 已在 Supabase 执行，29 张表已创建
- ☒ RLS 已临时关闭
- ☒ `src/types/database.ts` 已自动生成
- ☒ `sql/003_seed_data.sql` 已执行，演示数据已就绪

第四章 通用组件与布局搭建

4.1 侧边栏 Sidebar 组件

侧边栏是 ERP 系统的导航中枢，包含 14 个模块的入口。我们先让 Trae 生成基础结构，再手动调整细节。

按 `Ctrl+I` 唤起 Builder，输入：

创建 `src/components/layout/sidebar.tsx` 文件，实现 ERP 系统的侧边栏导航。要求：

【结构】

- 顶部 Logo 区域：显示"RCA6.0 ERP"文字 Logo（蓝色 #3298cb）
- 中间导航列表：14 个导航项，每项包含 `lucide-react` 图标 + 中文名称
- 底部用户信息：显示当前用户头像 + 姓名 + 角色，点击弹出退出登录

【14 个导航项】

1. 工作台 (LayoutDashboard icon, 路径 /)
2. 客户管理 (Users icon, 路径 /customers)
3. 供应商管理 (Truck icon, 路径 /suppliers)
4. 产品管理 (Package icon, 路径 /products)
5. 询盘管理 (MessageSquare icon, 路径 /inquiries)
6. 报价管理 (FileText icon, 路径 /quotations)
7. 订单管理 (ShoppingCart icon, 路径 /orders)
8. 采购管理 (ClipboardList icon, 路径 /purchases)
9. 库存管理 (Warehouse icon, 路径 /inventory)
10. 发货计划 (Ship icon, 路径 /shipments)
11. 财务管理 (DollarSign icon, 路径 /finance)
12. 单证管理 (FolderOpen icon, 路径 /documents)
13. 操作日志 (History icon, 路径 /audit-logs)
14. 系统设置 (Settings icon, 路径 /settings)

【样式】

- 宽度 220px，固定在左侧
- 背景色 #364e5b（深蓝灰）
- 文字白色，hover 时背景变浅 #435c69
- 当前激活项左侧有 3px 蓝色 #3298cb 竖条
- 用 `next/link` 和 `usePathname` 实现路由跳转和激活态

【响应式】

- 桌面端 (≥1024px) 默认展开
- 移动端 (<1024px) 默认收起，点击汉堡按钮展开
- 用 `useState` + `Tailwind` 类名控制显隐

【完整代码要求】

请生成完整的可运行代码，包括所有 `import` 语句，不要省略。组件需要标注 `"use client"`。

Trae 生成代码后，可能需要修复一些小问题。常见问题：

- **导入路径错误**：检查 `from "@components/ui/..."` 是否正确
- **图标不存在**：`lucide-react` 中部分图标名不同，让 Trae 替换为存在的图标
- **`usePathname` 报错**：确保文件顶部有 `"use client"` 声明

4.2 顶栏 Header 组件

按 `Ctrl+I` 唤起 Builder，输入：

创建 `src/components/layout/header.tsx` 文件，实现 ERP 系统的顶部栏。要求：

【结构】

- 左侧：汉堡按钮（移动端展开侧边栏）+ 当前页面标题（用 `usePathname` 推断）
- 中间：全局搜索框（占位符“全局搜索...”，`Ctrl+K` 唤起快捷搜索）
- 右侧：通知铃铛（带红点数字）+ 用户头像下拉菜单（含个人信息、退出登录）

【样式】

- 高度 60px，背景白色，底部 1px 灰色边框
- 搜索框宽度 400px，圆角 6px
- 通知铃铛 hover 显示通知列表（暂时用 mock 数据）
- 用户头像 32x32 圆形

【交互】

- 搜索框聚焦时显示快捷键提示“按 Enter 搜索”
- 通知铃铛点击跳转到 `/notifications`（暂时可以预留）
- 用户下拉菜单用 `shadcn DropdownMenu` 组件

【页面标题映射】

根据 `usePathname` 返回的路径，显示对应的中文标题：

- `"/` → “工作台”
- `"/customers"` → “客户管理”
- `"/suppliers"` → “供应商管理”
- `"/products"` → “产品管理”
- `"/inquiries"` → “询盘管理”
- `"/quotations"` → “报价管理”
- `"/orders"` → “订单管理”
- `"/purchases"` → “采购管理”
- `"/inventory"` → “库存管理”
- `"/shipments"` → “发货计划”
- `"/finance"` → “财务管理”
- `"/documents"` → “单证管理”
- `"/audit-logs"` → “操作日志”
- `"/settings"` → “系统设置”

导出默认组件，必须是 `"use client"` 组件。请生成完整代码。

4.3 主布局 Layout 组装

现在把侧边栏和顶栏组装成完整的应用布局。按 `Ctrl+I` 唤起 Builder，输入：

修改 `src/app/(dashboard)/layout.tsx` 文件，组装 ERP 主布局。

【要求】

1. 引入 `Sidebar` 和 `Header` 组件
2. 布局结构：


```
<div class="flex h-screen">
  <Sidebar />
  <div class="flex-1 flex flex-col">
    <Header />
    <main class="flex-1 overflow-auto p-6 bg-gray-50">{children}</main>
  </div>
</div>
```
3. 这个 `layout.tsx` 是 Server Component，但 `Sidebar` 和 `Header` 是 Client Component，可以直接引入
4. 确保根布局 `src/app/layout.tsx` 中导入了 `globals.css` 和 `Toaster`（来自 `sonner`）

同时修改 `src/app/(dashboard)/page.tsx`（即工作台首页），暂时显示一个简单的欢迎卡片：

```
<h1>欢迎来到 RCA6.0 ERP</h1><p>系统正在搭建中...</p>
```

完成后启动 `pnpm dev`，访问 `http://localhost:3000` 应该能看到带侧边栏和顶栏的主界面。

启动 `pnpm dev` 后访问 `http://localhost:3000`。如果一切正常，你会看到：左侧深色侧边栏带 14 个导航项，右侧顶部白栏带搜索框和头像，主区域显示欢迎文字。如果报错，把控制台错误信息复制给 Trae Chat 让它修复。

4.4 通用组件：DataTable

DataTable 是 ERP 中最重要的通用组件——所有列表页都要用它。它需要支持：分页、排序、搜索、筛选、批量选择、自定义列渲染。按 **Ctrl+I** 唤起 Builder，输入：

创建 `src/components/common/data-table.tsx` 文件，实现一个泛型表格组件 `GenericDataTable<T>`。

```
【Props 定义】
interface Column<T> {
  key: string;
  header: string;
  render?: (row: T) => React.ReactNode; // 自定义渲染
  sortable?: boolean;
  width?: string;
}

interface DataTableProps<T> {
  data: T[];
  columns: Column<T>[];
  loading?: boolean;
  pagination?: { page: number; pageSize: number; total: number; onPageChange: (page: number) => void };
  selectable?: boolean;
  onSelectionChange?: (selectedIds: string[]) => void;
  onSort?: (key: string, order: "asc" | "desc") => void;
  emptyState?: React.ReactNode;
  rowKey?: (row: T) => string; // 默认用 row.id
}
```

- 【功能要求】
1. 表头：深灰背景 #364e5b 白色文字，sortable 列显示排序图标 (ArrowUpDown)
 2. 表体：奇数行白色，偶数行 #eeeeff0，hover 时 #eaf4fa
 3. 复选框列 (selectable=true 时)：表头全选/反选，每行单选
 4. 分页：底部显示"共 X 条"，页码切换，每页条数选择 (10/20/50)
 5. 加载状态：用 Skeleton 行占位
 6. 空状态：居中显示空状态组件
 7. 响应式：宽度超出时横向滚动

- 【样式】
- 用 shadcn Table 组件作为基础
 - 复选框用 shadcn Checkbox
 - 分页用 shadcn Button
 - TypeScript 严格模式，不要用 any

【完整代码】
请生成完整代码，类型定义和组件实现都要完整。包括所有 import 语句。

4.5 其他通用组件

除了 DataTable，还需要几个小通用组件。可以一次性让 Trae 生成。按 **Ctrl+I** 唤起 Builder，输入：

在 `src/components/common/` 目录下创建以下组件：

1. `search-input.tsx` - 搜索输入框
 - Props: `value`, `onChange`, `placeholder`, `onEnter`
 - 内置 Search icon 和清除按钮 (X icon)
 - `debounce 300ms` 自动触发 `onChange`
 - 支持 `Ctrl+K` 全局聚焦 (用 `ref` + `useEffect` 监听键盘)
2. `filter-bar.tsx` - 筛选栏
 - 横向排列多个筛选器 (下拉、日期范围等)
 - 右侧"重置"按钮
 - 用 `flex` 布局, 自动换行
3. `status-badge.tsx` - 状态标签
 - Props: `status`, `variant`
 - `variant` 映射: `pending`=灰, `processing/confirmed`=蓝, `shipped`=青, `completed`=绿, `cancelled/rejected/overdue`=红, `expired`=橙
 - 用 `shadcn Badge` 的 `variant` 系统
4. `confirm-dialog.tsx` - 确认弹窗
 - Props: `open`, `title`, `description`, `confirmText`, `cancelText`, `onConfirm`, `onCancel`, `variant` (`default` | `destructive`)
 - 基于 `shadcn Dialog`
 - `destructive` 变体确认按钮为红色
5. `empty-state.tsx` - 空状态
 - Props: `icon`, `title`, `description`, `action` (`ReactNode`)
 - 居中显示, 灰色风格
6. `loading-skeleton.tsx` - 骨架屏
 - 导出多个变体: `TableSkeleton`, `CardSkeleton`, `FormSkeleton`
 - 用 `shadcn Skeleton` 组合实现

所有组件用 `TypeScript` 严格类型, 导出默认组件。

4.6 工具函数库

最后创建工具函数库 `lib/utils.ts`, 集中放置通用工具函数。按 `Ctrl+I` 唤起 Builder, 输入:

在 `src/lib/utils.ts` 中实现以下工具函数:

1. `cn(...inputs)` - 类名合并 (`clsx` + `tailwind-merge`)
完整实现:

```
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}
```

2. `formatCurrency(amount: number, currency: string)` - 格式化金额
 - 示例: `formatCurrency(56800, "USD")` → "\$56,800.00"
 - 示例: `formatCurrency(56800, "CNY")` → "¥56,800.00"
 - 用 `Intl.NumberFormat` 实现
3. `formatDate(date: string | Date, format?: string)` - 格式化日期
 - 默认格式 "YYYY-MM-DD"
 - 支持 "YYYY-MM-DD HH:mm:ss" 等
 - 用 `date-fns` 实现
4. `generateCode(prefix: string, date?: Date)` - 生成业务编码
 - 格式: `prefix + YYYYMMDD + 4位序号`
 - 示例: `generateCode("CU")` → "CU202607290001"
5. `downloadExcel(data: any[], filename: string)` - 下载 Excel
 - 用 `xlsx` 库生成 `.xlsx` 文件并触发下载
6. `downloadPDF(html: string, filename: string)` - 下载 PDF
 - 用 `jspdf + html2canvas` 生成
7. `debounce<T>(fn: T, delay: number)` - 防抖函数
8. `throttle<T>(fn: T, delay: number)` - 节流函数
9. `deepClone<T>(obj: T): T` - 深拷贝
10. `getInitials(name: string)` - 获取姓名首字母（用于头像）

所有函数都要有完整的 TypeScript 类型注解和 JSDoc 注释。

4.7 认证中间件

最后创建认证中间件，保护所有 `/dashboard` 路径，未登录用户跳转到 `/login`。按 `Ctrl+I` 唤起 Builder，输入：

创建 `src/middleware.ts` 文件，实现 Next.js 中间件：

【逻辑】

1. 拦截所有路径，除了 `/login`, `/api/auth/*`, `/_next/*`, `/favicon.ico`
2. 检查请求的 `cookie` 中是否有 Supabase 认证 `token`
3. 如果没有 → 重定向到 `/login?from=原始路径`
4. 如果有 → 放行

【完整代码模板】

```
import { NextResponse } from "next/server";
import { createServerClient } from "@supabase/ssr";
import type { NextRequest } from "next/server";

export async function middleware(request: NextRequest) {
  const response = NextResponse.next();

  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() {
```

```

return request.cookies.getAll();
},
setAll(cookiesToSet) {
cookiesToSet.forEach(({ name, value, options }) => {
response.cookies.set(name, value, options);
});
},
},
}
);

const { data: { session } } = await supabase.auth.getSession();

// 如果未登录且不在白名单路径，跳转到登录页
const isLoginPath = request.nextUrl.pathname.startsWith("/login");
const isApiAuthPath = request.nextUrl.pathname.startsWith("/api/auth");

if (!session && !isLoginPath && !isApiAuthPath) {
const redirectUrl = new URL("/login", request.url);
redirectUrl.searchParams.set("from", request.nextUrl.pathname);
return NextResponse.redirect(redirectUrl);
}

return response;
}

export const config = {
matcher: ["/((?!login|api/auth|_next/static|_next/image|favicon.ico).*)"],
};

```

请补全完整实现。同时创建 `src/app/(auth)/login/page.tsx` 登录页面：

- 居中显示登录卡片，含邮箱和密码输入框
- 用 `react-hook-form` + `zod` 校验
- 提交时调用 `/api/auth/login` 接口
- 登录成功后跳转到 `from` 参数指定的路径或 `/`

同时创建 `src/app/api/auth/login/route.ts` 和 `logout/route.ts` API：

- `POST /api/auth/login`: 接收 `{email, password}`，用 `supabase.auth.signInWithPassword` 登录，返回 `{user, message}`
- `POST /api/auth/logout`: 调用 `supabase.auth.signOut()`，返回 `{message}`
- `GET /api/auth/me`: 返回当前登录用户信息

请生成所有文件的完整代码。

完成检查清单：

- ☒ 访问 `http://localhost:3000` 自动跳转到 `/login` 登录页
- ☒ 登录后看到带侧边栏+顶栏的主界面
- ☒ 侧边栏 14 个导航项可点击跳转
- ☒ `src/components/common/` 下有 `DataTable`、`SearchInput`、`FilterBar`、`StatusBadge`、`ConfirmDialog`、`EmptyState`、`LoadingSkeleton`
- ☒ `src/lib/utils.ts` 有完整工具函数
- ☒ `src/middleware.ts` 已配置认证

第五章 客户管理模块（完整开发演示）

从本章开始，我们正式进入业务模块开发。客户管理是 ERP 中最基础也最典型的模块——它包含了“列表 + 表单 + 详情 + API”的完整 CRUD 套路。我会用最详尽的步骤演示如何用 Trae 完成客户管理的开发，把这个套路彻底吃透。后续模块（产品、供应商等）套路相同，只会重点讲差异点。

5.1 需求分析与字段设计

在写代码之前，先想清楚要做什么。客户管理模块的核心需求：

- **列表展示**：分页显示所有客户，支持搜索、筛选、排序
- **新建客户**：通过弹窗表单创建新客户
- **编辑客户**：修改已有客户信息
- **删除客户**：软删除（设置 `deleted_at`），不真正删除
- **查看详情**：跳转到详情页，显示客户基本信息 + 关联的询盘/订单
- **导入导出**：支持 Excel 批量导入和导出

字段设计参照第三章建表时的 `customers` 表：编码、名称、英文名、联系人、邮箱、电话、国家、地址、等级、状态、信用额度、币种、付款条件、负责业务员、备注、时间戳。

5.2 用 Trae Builder 一次性生成模块代码

现在让 Trae 一次性生成客户管理模块的所有文件。这是 Trae Builder 最擅长的场景。按 **Ctrl+I** 唤起 Builder，输入：

请帮我生成客户管理模块的完整代码，包含以下文件：

```
=== 文件 1: src/types/models.ts ===
定义 Customer 类型（与数据库 customers 表对应）：
{ id, code, name, name_en, contact_person, contact_email, contact_phone, country, address, website, level:
"normal"|"vip"|"strategic", status: "active"|"silent"|"lost", credit_limit, payment_terms, preferred_currency, notes,
assigned_to, created_at, updated_at }

=== 文件 2: src/lib/code-generator.ts ===
实现 generateCode(table, prefix) 函数：
- 查询指定表中今日已存在的编码数量
- 生成下一个编码：prefix + YYYYMMDD + 4位序号
- 用 Supabase 服务端客户端
- 示例：generateCode("customers", "CU") → "CU202607290001"
完整代码：
```

```
import { createClient } from "@lib/supabase/server";
```

```

export async function generateCode(table: string, prefix: string): Promise<string> {
  const supabase = createClient();
  const today = new Date();
  const dateStr = today.getFullYear().toString() +
    String(today.getMonth() + 1).padStart(2, "0") +
    String(today.getDate()).padStart(2, "0");

  const pattern = `${prefix}${dateStr}%`;
  const { count } = await supabase
    .from(table)
    .select("*", { count: "exact", head: true })
    .like("code", pattern);

  const seq = String((count ?? 0) + 1).padStart(4, "0");
  return `${prefix}${dateStr}${seq}`;
}

export async function generateFormattedCode(table: string, prefix: string): Promise<string> {
  const supabase = createClient();
  const year = new Date().getFullYear();
  const pattern = `${prefix}-${year}-`;
  const { count } = await supabase
    .from(table)
    .select("*", { count: "exact", head: true })
    .like("code", pattern);

  const seq = String((count ?? 0) + 1).padStart(4, "0");
  return `${prefix}-${year}-${seq}`;
}

```

```

=== 文件 3: src/lib/audit.ts ===
实现审计日志工具：

```

```

import { createClient } from "@lib/supabase/server";

interface AuditParams {
  user_id: string;
  module: string;
  action: "create" | "update" | "delete" | "approve" | "reject" | "status_change";
}

```

```

record_id?: string;
record_code?: string;
before_data?: any;
after_data?: any;
description: string;
}

export async function logAction(params: AuditParams) {
const supabase = createClient();
await supabase.from("audit_logs").insert(params);
}

```

```

=== 文件 4: src/app/api/customers/route.ts ===
GET /api/customers - 列表查询
- 参数: page, pageSize, search, level, status, country, sortBy, sortOrder
- 返回 { data, total, page, pageSize }
- 用 ilike 模糊搜索 name/code/contact_person
- 用 Supabase 服务端客户端

POST /api/customers - 创建
- 用 zod 校验 body
- 自动生成 code (用 generateCode("customers", "CU"))
- 自动设置 created_at, updated_at
- 用 logAction 记录创建操作
- 返回 { data, message }

```

完整代码：

```

import { NextRequest, NextResponse } from "next/server";
import { createClient } from "@/lib/supabase/server";
import { generateCode } from "@/lib/code-generator";
import { logAction } from "@/lib/audit";
import { z } from "zod";

const customerSchema = z.object({
name: z.string().min(1).max(200),
name_en: z.string().max(200).optional(),
contact_person: z.string().min(1),
contact_email: z.string().email().optional().or(z.literal("")),
contact_phone: z.string().optional(),
country: z.string().min(1),
address: z.string().optional(),
website: z.string().optional(),

```

```
level: z.enum(["normal", "vip", "strategic"]).default("normal"),
status: z.enum(["active", "silent", "lost"]).default("active"),
credit_limit: z.number().min(0).default(0),
payment_terms: z.string().default("T/T"),
preferred_currency: z.string().default("USD"),
notes: z.string().optional(),
});

export async function GET(request: NextRequest) {
  const searchParams = request.nextUrl.searchParams;
  const page = Number(searchParams.get("page") ?? 1);
  const pageSize = Number(searchParams.get("pageSize") ?? 20);
  const search = searchParams.get("search") ?? "";
  const level = searchParams.get("level");
  const status = searchParams.get("status");

  const supabase = createClient();

  let query = supabase
    .from("customers")
    .select("*", { count: "exact" })
    .is("deleted_at", null);

  if (search) {
    query = query.or(name.ilike.%${search}%,code.ilike.%${search}%,contact_person.ilike.%${search}%);
  }

  if (level) query = query.eq("level", level);
  if (status) query = query.eq("status", status);

  const sortBy = searchParams.get("sortBy") ?? "created_at";
  const sortOrder = searchParams.get("sortOrder") === "asc" ? false : true;
  query = query.order(sortBy, { ascending: !sortOrder });

  const { data, count, error } = await query
    .range((page - 1) * pageSize, page * pageSize - 1);

  if (error) return NextResponse.json({ error: error.message }, { status: 500 });

  return NextResponse.json({ data: data ?? [], total: count ?? 0, page, pageSize });
}
```



```
export async function POST(request: NextRequest) {
  const body = await request.json();
  const validated = customerSchema.parse(body);
  const code = await generateCode("customers", "CU");
  const supabase = createClient();

  const { data, error } = await supabase
    .from("customers")
    .insert({ ...validated, code })
    .select()
    .single();

  if (error) return NextResponse.json({ error: error.message }, { status: 500 });

  // 写审计日志
  await logAction({
    user_id: "system", // TODO: 从 session 获取
    module: "customers",
    action: "create",
    record_id: data.id,
    record_code: data.code,
    after_data: data,
    description: 创建客户 ${data.name} (${data.code}),
  });

  return NextResponse.json({ data, message: "创建成功" }, { status: 201 });
}
```

```

=== 文件 5: src/app/api/customers/[id]/route.ts ===
GET /api/customers/[id] - 详情
- 返回单个客户信息
- 同时返回关联的询盘数、订单数

PUT /api/customers/[id] - 更新
- 用 zod 校验
- 先查旧数据，更新后查新数据
- 用 logAction 记录变更（含 before_data, after_data）

DELETE /api/customers/[id] - 软删除
- 只设置 deleted_at = now()
- 用 logAction 记录删除

=== 文件 6: src/components/customers/customer-form.tsx ===
客户表单弹窗组件（用于新建和编辑）
- Props: open, onChange, customer?(编辑时传入), onSuccess
- 用 react-hook-form + zodResolver
- 字段: name(必填), name_en, contact_person(必填), contact_email, contact_phone, country(必填), address, website,
level(下拉), status(下拉), credit_limit(数字), payment_terms, preferred_currency(下拉), notes(textarea)
- 校验: email 格式, phone 至少 6 位, credit_limit ≥ 0
- 提交时调用 POST /api/customers 或 PUT /api/customers/[id]
- 用 @tanstack/react-query 的 useMutation
- 成功后 toast 提示并关闭弹窗
- 加载状态显示 disabled 按钮
- 用 shadcn Dialog, Form, Input, Select, Button, Textarea 组件

=== 文件 7: src/app/(dashboard)/customers/page.tsx ===
客户管理列表页 ("use client")
- 顶部: 标题"客户管理" + 工具栏（导出、导入、新建客户按钮）
- 筛选栏: 搜索框 + 等级下拉 + 状态下拉 + 国家输入 + 创建日期范围
- 表格: 复选框, 编码, 客户名称, 联系人, 国家, 等级(彩色badge), 状态(彩色badge), 创建时间, 操作列
- 操作列: 查看(跳详情)、编辑(打开表单)、删除(确认弹窗) 三个图标按钮
- 分页: 用 GenericDataTable 自带的分页
- 用 useQuery 获取数据
- 删除用 ConfirmDialog 二次确认

=== 文件 8: src/app/(dashboard)/customers/[id]/page.tsx ===

```

客户详情页

- 顶部: 返回按钮 + 客户名称 + 编辑按钮
- 上半部分: 基本信息卡片（左右两列展示所有字段）
- 下半部分: Tab 切换显示"询盘记录""报价记录""订单记录""收款记录"
- 每个 Tab 调用对应 API 拉取关联数据

【代码质量要求】

- 所有组件用 TypeScript 严格类型
- API 路由统一错误处理
- 引入路径用 @/ 别名
- 不要省略任何代码，包括 import 语句
- 注释关键业务逻辑

请按上述顺序生成所有 8 个文件的完整代码。

Trae 会逐个创建这 8 个文件。这个过程可能需要 30-60 秒，期间会显示"Thinking..."和文件创建进度。**不要中途打断**。完成后 Trae 会在右下角显示"已创建 X 个文件"的提示。

5.3 列表页代码详解

打开 `src/app/(dashboard)/customers/page.tsx` 查看 Trae 生成的代码。重点理解以下几个部分：

5.3.1 数据获取

列表页用 @tanstack/react-query 的 useQuery hook 获取数据。核心代码：

```
const { data, isLoading } = useQuery({
  queryKey: ["customers", { page, pageSize, search, level, status }],
  queryFn: async () => {
    const params = new URLSearchParams({
      page: String(page),
      pageSize: String(pageSize),
      search, level, status,
    });
    const res = await fetch(`/api/customers?${params}`);
    if (!res.ok) throw new Error("加载失败");
    return res.json();
  },
});
```

queryKey 是缓存的键，参数变化时会自动重新请求。queryFn 是实际的请求函数。isLoading 是加载状态。

5.3.2 筛选与搜索联动

每个筛选条件变化时都要重置 page 为 1:

```
const [filters, setFilters] = useState({
  search: "", level: "", status: "", country: "",
});
const [page, setPage] = useState(1);

useEffect(() => {
  setPage(1);
}, [filters]);
```

5.4 表单弹窗代码详解

打开 `src/components/customers/customer-form.tsx`。关键点：

5.4.1 Zod Schema 定义

```
const customerSchema = z.object({
  name: z.string().min(1, "客户名称不能为空").max(200),
  name_en: z.string().max(200).optional(),
  contact_person: z.string().min(1, "联系人不能为空"),
  contact_email: z.string().email("邮箱格式不正确").optional().or(z.literal("")),
  contact_phone: z.string().min(6, "电话至少 6 位").optional().or(z.literal("")),
  country: z.string().min(1, "国家不能为空"),
});
type CustomerFormValues = z.infer<typeof customerSchema>;
```

5.4.2 提交逻辑

```
const mutation = useMutation({
  mutationFn: async (values: CustomerFormValues) => {
    if (customer) {
      const res = await fetch(`/api/customers/${customer.id}`, {
        method: "PUT",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(values),
      });
      if (!res.ok) throw new Error("更新失败");
    } else {
      const res = await fetch("/api/customers", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(values),
      });
      if (!res.ok) throw new Error("创建失败");
    }
  },
  onSuccess: () => {
    toast.success(customer ? "更新成功" : "创建成功");
    queryClient.invalidateQueries({ queryKey: ["customers"] });
    onOpenChange(false);
    onSuccess?.();
  },
  onError: (err) => {
    toast.error(err.message);
  },
});
```

5.5 测试与调试

代码生成后，启动

pnpm

dev，访问

http://localhost:3000/customers。第一次访问可能会遇到各种报错，按以下顺序排查：

报错信息	原因	解决方法
Module not found: @/components/ui/xxx	shadcn 组件未安装	npx shadcn-ui@latest add xxx
Supabase connection refused	.env.local 配置错误	检查 URL 和 ANON_KEY
RLS blocked the query	RLS 未关闭	在 Supabase SQL Editor 执行 3.5 节的 SQL
Type error: Property 'xxx' does not exist	类型不匹配	让 Trae Chat 检查 src/types/database.ts
Hydration mismatch	服务端和客户端渲染不一致	在 useQuery 前加 "use client" 声明
CORS error	前端跨域请求被拒	确保 API 路径以 /api 开头，同源访问

调试技巧：遇到报错时，把完整的错误堆栈复制到 Trae Chat (Ctrl+L)，它会给出修复建议。如果 Chat 给的修复没生效，把当前文件内容一起粘贴过去，让 Trae 基于上下文修复。

5.6 功能验证清单

客户管理模块跑通后，按以下清单逐项验证：

- ☒ 列表页能加载演示数据（5 条客户）
- ☒ 搜索框输入文字后列表实时筛选
- ☒ 等级下拉筛选生效
- ☒ 状态下拉筛选生效
- ☒ 点击表头能切换排序

- ☒ 翻页按钮可点击
- ☒ 点击"新建客户"按钮弹出表单弹窗
- ☒ 表单校验生效（必填项为空时提交报错）
- ☒ 提交成功后弹窗关闭，列表自动刷新
- ☒ 点击行的编辑图标能预填数据
- ☒ 点击行的查看图标跳转到详情页
- ☒ 详情页显示基本信息
- ☒ 详情页的关联询盘/订单 Tab 显示数据
- ☒ 点击删除图标弹出确认弹窗
- ☒ 确认删除后列表刷新且记录消失
- ☒ 在 Supabase Table Editor 检查 audit_logs 表，应有对应记录

本章总结：客户管理模块的完整开发流程是后续所有模块的模板：① 需求分析 → ② 让 Trae 一次性生成所有文件 → ③ 启动验证 → ④ 把报错反馈给 Trae 修复 → ⑤ 按清单逐项验证功能。后续模块都按这个流程走，只是 Prompt 内容不同。

第六章 产品管理模块

产品管理模块在客户管理的基础上增加了几个特殊点：多图上传、分类树、OEM 号搜索、产品-供应商多对多关联。本章重点讲这些差异点，相同的 CRUD 部分不再重复。

6.1 多图上传（Supabase Storage）

产品图片需要上传到 Supabase Storage（对象存储服务）。配置步骤：

步骤 1：创建 Storage Bucket

在 Supabase Dashboard 左侧点击"Storage" → "New bucket"，名称填"product-images"，勾选"Public bucket"（允许公开访问），点击"Create"。

步骤 2：创建上传 API

按 **Ctrl+I** 唤起 Builder，输入：

创建 `src/app/api/products/[id]/images/route.ts` 实现：

```
POST /api/products/[id]/images
- 接收 multipart/form-data, 字段名 file
- 校验文件类型 (jpg/png/webp) 和大小 (<2MB)
- 生成唯一文件名: {product_id}/{timestamp}-{random}.ext
- 上传到 Supabase Storage 的 product-images bucket
- 获取 public URL
- 把 URL 追加到 products.image_urls 数组 (用 supabase 的 array_append)
- 返回 { url, message }

DELETE /api/products/[id]/images?index=X
- 从 image_urls 数组中移除第 X 个 URL
- 同时从 Storage 删除文件
- 返回 { message }
```

【完整代码】

```
import { NextRequest, NextResponse } from "next/server";
import { createClient } from "@lib/supabase/server";
import { createAdminClient } from "@lib/supabase/admin";

export async function POST(
  request: NextRequest,
  { params }: { params: { id: string } }
) {
  const formData = await request.formData();
  const file = formData.get("file") as File;

  if (!file) {
    return NextResponse.json({ error: "未选择文件" }, { status: 400 });
  }

  // 校验文件类型
  const allowedTypes = ["image/jpeg", "image/png", "image/webp"];
  if (!allowedTypes.includes(file.type)) {
    return NextResponse.json({ error: "仅支持 JPG/PNG/WEBP 格式" }, { status: 400 });
  }

  // 校验文件大小
  if (file.size > 2 * 1024 * 1024) {
    return NextResponse.json({ error: "文件大小不能超过 2MB" }, { status: 400 });
  }

  const ext = file.name.split(".").pop();
  const fileName = `${params.id}/${Date.now()}-${Math.random().toString(36).slice(2)}.${ext}`;

  const adminClient = createAdminClient();
  const { error: uploadError } = await adminClient.storage
    .from("product-images")
    .upload(fileName, file);

  if (uploadError) {
    return NextResponse.json({ error: uploadError.message }, { status: 500 });
  }

  const { data: urlData } = adminClient.storage
```

```
.from("product-images")
.getPublicUrl(fileName);

const url = urlData.publicUrl;

// 把 URL 追加到 image_urls 数组

const supabase = createClient();

const { data: product } = await supabase
  .from("products")
  .select("image_urls")
  .eq("id", params.id)
  .single();

const currentUrls = product?.image_urls ?? [];
const newUrls = [...currentUrls, url];

await supabase
  .from("products")
  .update({ image_urls: newUrls, updated_at: new Date().toISOString() })
  .eq("id", params.id);

return NextResponse.json({ url, message: "上传成功" });
}

export async function DELETE(
  request: NextRequest,
  { params }: { params: { id: string } }
) {
  const index = Number(request.nextUrl.searchParams.get("index"));

  const supabase = createClient();
  const adminClient = createAdminClient();

  const { data: product } = await supabase
    .from("products")
    .select("image_urls")
    .eq("id", params.id)
    .single();

  const currentUrls = product?.image_urls ?? [];
```

```
const urlToRemove = currentUrls[index];
if (!urlToRemove) {
  return NextResponse.json({ error: "索引无效" }, { status: 400 });
}

// 从 Storage 删除
const path = urlToRemove.split("/product-images/")[1];
if (path) {
  await adminClient.storage.from("product-images").remove([path]);
}

// 从数组中移除
const newUrls = currentUrls.filter((_: string, i: number) => i !== index);
await supabase
  .from("products")
  .update({ image_urls: newUrls, updated_at: new Date().toISOString() })
  .eq("id", params.id);

return NextResponse.json({ message: "删除成功" });
}
```

同时创建前端组件 `src/components/products/image-uploader.tsx`:

- 支持拖拽上传和点击上传
- 显示已上传图片缩略图网格（每行 4 张，正方形）
- 每张图右上角有删除按钮（hover 显示）
- 上传中显示进度条
- 用 `react-dropzone` 库（`pnpm add react-dropzone`）

请生成完整代码。

6.2 分类树形结构

产品分类支持多级（如“制动系统 → 刹车片 → 陶瓷刹车片”）。让 Trae 生成树形选择器组件：

创建 `src/components/products/category-tree-select.tsx` 组件：

- Props: `value`, `onChange`
- 用 `shadcn Popover` + 自定义树形列表实现
- 调用 `GET /api/products/categories` 获取所有分类
- 在前端构建树形结构（用递归）
- 显示为带缩进的下拉列表，每级缩进 16px
- 支持搜索分类名

同时创建 `src/app/api/products/categories/route.ts`：

- GET - 返回所有分类（扁平数组）
- POST - 创建新分类（`name`, `parent_id`）

【GET 完整代码】


```
import { NextResponse } from "next/server";
import { createClient } from "@/lib/supabase/server";

export async function GET() {
  const supabase = createClient();
  const { data, error } = await supabase
    .from("product_categories")
    .select("*")
    .order("sort_order");

  if (error) return NextResponse.json({ error: error.message }, { status: 500 });
  return NextResponse.json({ data: data ?? [] });
}

export async function POST(request: Request) {
  const body = await request.json();
  const supabase = createClient();
  const { data, error } = await supabase
    .from("product_categories")
    .insert(body)
    .select()
    .single();

  if (error) return NextResponse.json({ error: error.message }, { status: 500 });
  return NextResponse.json({ data, message: "创建成功" }, { status: 201 });
}
```

6.3 OEM 号精确搜索

汽配行业的 OEM 号是核心搜索字段。在列表页搜索框旁加一个"精确匹配"开关：

```
const [exactMatch, setExactMatch] = useState(false);

// 查询时根据 exactMatch 选择 eq 或 ilike
if (search) {
  if (exactMatch) {
    query = query.eq("oem_number", search);
  } else {
    query = query.or(`name.ilike.${search}%,code.ilike.${search}%,oem_number.ilike.${search}%`);
  }
}
```

6.4 产品-供应商多对多关联

一个产品可能有多个供应商供货。在产品表单中可以选择多个供应商：

在产品表单中添加"供应商关联"区域：

- 显示已关联的供应商列表（表格：供应商名称、是否主供应商、供货价、最小起订量、交期）
- 点击"添加供应商"按钮弹出选择器
- 供应商选择器：搜索下拉，选择后添加一行
- 每行可以编辑供货价、最小起订量、交期、是否主供应商（单选，只能有一个主）
- 每行有删除按钮
- 提交时一并保存到 product_suppliers 表（先删除该产品的所有关联，再批量插入）

创建 src/app/api/products/[id]/suppliers/route.ts：
GET - 返回该产品的所有供应商关联（含供应商名称 join）
PUT - 接收 suppliers 数组，替换该产品的所有关联

【PUT 完整代码】

```
import { NextRequest, NextResponse } from "next/server";
import { createClient } from "@/lib/supabase/server";

export async function PUT(
  request: NextRequest,
  { params }: { params: { id: string } }
) {
  const body = await request.json();
  const supabase = createClient();

  // 1. 先删除该产品的所有供应商关联
  await supabase
    .from("product_suppliers")
    .delete()
    .eq("product_id", params.id);

  // 2. 批量插入新关联
  if (body.suppliers?.length > 0) {
    const items = body.suppliers.map((s: any) => ({
      product_id: params.id,
      supplier_id: s.supplier_id,
      is_primary: s.is_primary || false,
      supplier_sku: s.supplier_sku,
      supply_price: s.supply_price,
      min_order_qty: s.min_order_qty,
      lead_time_days: s.lead_time_days,
```

```
));  
  
const { error } = await supabase.from("product_suppliers").insert(items);  
if (error) return NextResponse.json({ error: error.message }, { status: 500 });  
  
}  
  
return NextResponse.json({ message: "保存成功" });  
  
}
```

第七章 供应商管理模块

供应商管理与客户管理在数据结构上几乎对称（都是基础信息 CRUD），区别在于字段语义和业务关联。本章快速过一遍重点差异。

7.1 字段差异

相比客户，供应商独有的字段：

- 主营品类（main_category）：用多选标签，如"制动系统, 发动机配件"
- 平均交期（lead_time_days）：整数天，用于采购计算
- 质量评级（quality_rating）：1-5 星，用星级组件展示
- 合作状态（status）：active / blacklisted（黑名单）

7.2 完整模块生成 Prompt

按 **Ctrl+I** 唤起 Builder，输入：

参照客户管理模块的结构（src/app/(dashboard)/customers/ 和 src/app/api/customers/），生成完整的供应商管理模块。差异点：

1. 编码前缀用 SU（如 SU202607290001）
2. 字段：name, name_en, contact_person, contact_email, contact_phone, country, address, level(normal/preferred/strategic), status(active/blacklisted), main_category(text[]), lead_time_days(int), quality_rating(1-5), payment_terms, notes
3. 列表页表格列：编码、名称、联系人、国家、主营品类(用tag展示)、等级、评级(星)、状态、操作
4. 表单字段同上，质量评级用星级组件（lucide Star icon × 5）
5. 主营品类用多选输入（输入后回车添加为 tag）
6. 详情页含"供货产品""采购记录""应付记录"三个 Tab

请生成以下文件：

- src/types/models.ts 中追加 Supplier 类型
- src/app/api/suppliers/route.ts (GET, POST)
- src/app/api/suppliers/[id]/route.ts (GET, PUT, DELETE)
- src/app/api/suppliers/[id]/products/route.ts (GET) - 返回该供应商供货的所有产品
- src/components/suppliers/supplier-form.tsx
- src/components/suppliers/star-rating.tsx
- src/components/suppliers/category-multi-input.tsx
- src/app/(dashboard)/suppliers/page.tsx
- src/app/(dashboard)/suppliers/[id]/page.tsx

【star-rating.tsx 完整代码】

```
"use client";

import { Star } from "lucide-react";
import { useState } from "react";
import { cn } from "@/lib/utils";

interface StarRatingProps {
  value: number;
  onChange?: (value: number) => void;
  readOnly?: boolean;
  size?: number;
}

export function StarRating({ value, onChange, readOnly, size = 20 }: StarRatingProps) {
  const [hover, setHover] = useState(0);
  const stars = [1, 2, 3, 4, 5];

  return (
    <div className="flex items-center gap-1">
      {stars.map((star) => (
        <button
          key={star}
          type="button"
          disabled={readOnly}
          onClick={() => onChange?.(star)}
          onMouseEnter={() => !readOnly && setHover(star)}
          onMouseLeave={() => !readOnly && setHover(0)}
          className={cn("transition", !readOnly && "cursor-pointer")}
        >
          <Star
            size={size}
            className={cn(
              "transition",
              (hover || value) >= star
                ? "fill-yellow-400 text-yellow-400"
                : "text-gray-300"
            )}
          />
        </button>
      ))}
    </div>
  );
}
```

```
    )}
  />
</button>
  )}
</div>
);
}
```

请生成所有文件的完整代码，不要省略任何 import 语句。

第八章 询盘管理模块

询盘是外贸业务的起点，从这章开始业务流程变复杂——询盘包含明细（一对多），还能转报价单。本章重点讲一对多关系和“转报价”的跨界操作。

8.1 询盘与询盘明细的一对多关系

一个询盘可以包含多个产品询价。表结构：inquiries 主表 + inquiry_items 明细表。查询时用 Supabase 的关联查询一次性拿主表和明细：

```
const { data } = await supabase
  .from("inquiries")
  .select(`
    *,
    customer:customers(id, name, country),
    items:inquiry_items(
      id, product_id, product:products(id, name, oem_number),
      quantity, unit, target_price, notes
    )
  `)
  .eq("id", inquiryId)
  .single();
```

8.2 动态明细表格组件

询盘表单中需要一个“动态明细表格”——可以增删行、每行选择产品并输入数量。按 **Ctrl+I** 唤起 Builder，输入：

创建 `src/components/common/inquiry-items-editor.tsx` 组件：

【Props】

`items: InquiryItem[]`
`onChange: (items: InquiryItem[]) => void`

【功能】

1. 表格列：序号、产品(搜索选择器)、OEM号(自动带出)、数量(数字输入)、单位(下拉)、目标价(数字)、备注(输入)、操作(删除)
2. 底部"+ 添加行"按钮，点击添加空行
3. 产品选择器：用 Combobox (shadcn Popover + Command)，搜索产品名/OEM号
4. 选择产品后自动填充 OEM号和单位
5. 支持拖拽排序（用 `@dnd-kit/core` 库）
6. 每行有删除按钮
7. 数量和目标价实时计算"询盘总价值"显示在底部

【类型】

```
interface InquiryItem {
  id?: string;
  product_id: string;
  product_name?: string;
  oem_number?: string;
  quantity: number;
  unit: string;
  target_price?: number;
  notes?: string;
}
```

【完整代码要求】

请生成完整代码，包括所有 `import` 语句。同时创建产品搜索 API：
`src/app/api/products/search/route.ts`
`GET /api/products/search?q=keyword` → 返回 `[[id, name, code, oem_number, unit]]`

8.3 客户选择器组件

询盘表单需要选择客户，且支持"找不到客户时快速新建"。按 **Ctrl+I** 唤起 Builder，输入：

创建 `src/components/common/customer-select.tsx` 组件：

- Props: value, onChange
- 用 shadcn Popover + Command 实现 Combobox
- 输入时调用 `GET /api/customers?search=xxx&pageSize=10` 搜索
- 下拉列表显示客户名称 + 国家 + 等级 badge
- 列表底部有"+ 新建客户"按钮，点击弹出 `customer-form` 弹窗
- 新建成功后自动选中

【完整代码】

```
"use client";

import { useState } from "react";

import { Popover, PopoverContent, PopoverTrigger } from "@components/ui/popover";
import { Command, CommandEmpty, CommandGroup, CommandInput, CommandItem, CommandList } from "@components/ui/command";
import { Button } from "@components/ui/button";
import { Plus } from "lucide-react";
import { useQuery } from "@tanstack/react-query";
import { StatusBadge } from "./status-badge";

interface CustomerSelectProps {
```

```

value: string;
onChange: (value: string, label: string) => void;
}

export function CustomerSelect({ value, onChange }: CustomerSelectProps) {
const [open, setOpen] = useState(false);
const [search, setSearch] = useState("");

const { data } = useQuery({
  queryKey: ["customers-search", search],
  queryFn: async () => {
const res = await fetch(`/api/customers?search=${search}&pageSize=10`);
return res.json();
},
});

return (
  <Popover open={open} onOpenChange={setOpen}>
    <PopoverTrigger asChild>
      <Button variant="outline" role="combobox" className="w-full justify-between">
        {value ? "已选择客户" : "选择客户..."}
      </Button>
    </PopoverTrigger>
    <PopoverContent className="w-[400px] p-0" align="start">
      <Command>
        <CommandInput placeholder="搜索客户..." value={search} onChange={setSearch} />
        <CommandList>
          <CommandEmpty>未找到客户</CommandEmpty>
          <CommandGroup>
            {data?.data?.map((c: any) => (
              <CommandItem
                key={c.id}
                onSelect={() => {
                  onChange(c.id, c.name);
                  setOpen(false);

```

```
  }}  
  >  
  <div className="flex items-center gap-2 w-full">  
    <span>{c.name}</span>  
    <span className="text-xs text-gray-500">{c.country}</span>  
    <StatusBadge status={c.level} />  
  </div>  
</CommandItem>  
  )})  
</CommandGroup>  
</CommandList>  
<div className="border-t p-2">  
  <Button  
    variant="ghost"  
    size="sm"  
    className="w-full"  
    onClick={() => {  
      // 触发新建客户弹窗  
      setOpen(false);  
    }}  
  >  
    <Plus className="mr-2 h-4 w-4" />  
    新建客户  
  </Button>  
</div>  
</Command>  
</PopoverContent>  
</Popover>  
</>  
}
```

8.4 询盘状态机

询盘状态流转：pending → processing → quoted → closed / cancelled。按 **Ctrl+I** 唤起 Builder，输入：

创建 src/lib/inquiry-state-machine.ts：

```
const TRANSITIONS = {
  pending: ["processing", "cancelled"],
  processing: ["quoted", "cancelled"],
  quoted: ["closed", "cancelled"],
  closed: [],
  cancelled: [],
};

export function canTransition(from: string, to: string) {
  return TRANSITIONS[from]?.includes(to) ?? false;
}

export async function transitionInquiry(inquiryId, from, to, user) {
  // 1. 校验流转合法性
  if (!canTransition(from, to)) {
    throw new BusinessError(`不允许从 ${from} 流转到 ${to}`);
  }

  // 2. 更新 inquiries.status
  const supabase = createClient();
  const { error } = await supabase
    .from("inquiries")
    .update({ status: to, updated_at: new Date().toISOString() })
    .eq("id", inquiryId)
    .eq("status", from); // 乐观锁

  if (error) throw error;

  // 3. 写审计日志
  await logAction({
    user_id: user.id,
    module: "inquiries",
    action: "status_change",
    record_id: inquiryId,
    before_data: { status: from },
    after_data: { status: to },
    description: `询盘状态变更: ${from} → ${to}`,
  });
}
```

同时创建 API: src/app/api/inquiries/[id]/status/route.ts
POST 接收 { status, reason }, 调用 transitionInquiry

8.5 转报价单功能

询盘到报价的转换是关键业务动作。按 **Ctrl+I** 唤起 Builder，输入：

创建 API `src/app/api/inquiries/[id]/convert-to-quotation/route.ts`:
 POST 接收可选的 `body: { quotation_data?: Partial<Quotation> }`

【逻辑】

1. 查询询盘及其明细
2. 创建 quotations 记录:
 - `code = generateFormattedCode("quotations", "QT")`
 - `customer_id = inquiry.customer_id`
 - `inquiry_id = inquiry.id`
 - `pricing_status = "pending" // 待定价`
 - `status = "draft"`
 - `currency = 客户的 preferred_currency`
 - `valid_until = today + 30 天`
3. 批量创建 quotation_items:
 - 复制 `inquiry_items` 的 `product_id`, `quantity`, `unit`
 - `unit_price` 暂时为 0 (待业务员填写)
4. 更新 `inquiry.status = "quoted"`
5. 写审计日志
6. 返回新创建的 quotation 信息

【完整代码】

```
import { NextRequest, NextResponse } from "next/server";
import { createClient } from "@lib/supabase/server";
import { generateFormattedCode } from "@lib/code-generator";
import { logAction } from "@lib/audit";

export async function POST(
  request: NextRequest,
  { params }: { params: { id: string } }
) {
  const supabase = createClient();

  // 1. 查询询盘及明细
  const { data: inquiry, error: inqError } = await supabase
    .from("inquiries")
    .select(`
      *,
      customer:customers(preferred_currency),
      items:inquiry_items(*)
    `)
    .eq("id", params.id)
    .single();

  if (inqError || !inquiry) {
    return NextResponse.json({ error: "询盘不存在" }, { status: 404 });
  }
}
```

```
// 2. 生成报价编码
const code = await generateFormattedCode("quotations", "QT");
const validUntil = new Date();
validUntil.setDate(validUntil.getDate() + 30);

// 3. 创建报价单
const { data: quotation, error: quoteError } = await supabase
  .from("quotations")
  .insert({
    code,
    customer_id: inquiry.customer_id,
    inquiry_id: inquiry.id,
    pricing_status: "pending",
    status: "draft",
    currency: inquiry.customer?.preferred_currency || "USD",
    valid_until: validUntil.toISOString().split("T")[0],
    trade_terms: "FOB",
    payment_terms: "T/T",
  })
  .select()
  .single();

if (quoteError) return NextResponse.json({ error: quoteError.message }, { status: 500 });

// 4. 复制明细
if (inquiry.items?.length > 0) {
  const quoteItems = inquiry.items.map((item: any) => ({
    quotation_id: quotation.id,
    product_id: item.product_id,
    quantity: item.quantity,
    unit: item.unit,
    unit_price: 0, // 待业务员填写
    notes: item.notes,
  }));

  const { error: itemsError } = await supabase
```

```
.from("quotation_items")
.insert(quoteItems);

if (itemsError) return NextResponse.json({ error: itemsError.message }, { status: 500 });
}

// 5. 更新询盘状态
await supabase
.from("inquiries")
.update({ status: "quoted", updated_at: new Date().toISOString() })
.eq("id", params.id);

// 6. 写审计日志
await logAction({
  user_id: "system",
  module: "inquiries",
  action: "status_change",
  record_id: params.id,
  record_code: inquiry.code,
  after_data: { status: "quoted", quotation_id: quotation.id },
  description: `询盘 ${inquiry.code} 转为报价单 ${quotation.code}`,
});

return NextResponse.json({ data: quotation, message: "已转为报价单" });
}
```

第九章 报价管理模块

报价管理在询盘的基础上增加了定价流程、有效期管理、客户接受后转订单等环节。

9.1 完整模块生成 Prompt

按 **Ctrl+I** 唤起 Builder，输入：

生成完整的报价管理模块。请创建以下文件：

1. src/app/api/quotations/route.ts (GET 列表, POST 创建)
2. src/app/api/quotations/[id]/route.ts (GET, PUT, DELETE)
3. src/app/api/quotations/[id]/mark-priced/route.ts (POST - 标记已定价)
4. src/app/api/quotations/[id]/send/route.ts (POST - 发送给客户)
5. src/app/api/quotations/[id]/accept/route.ts (POST - 客户接受, 自动转订单)
6. src/app/api/quotations/[id]/reject/route.ts (POST - 客户拒绝)
7. src/app/api/quotations/[id]/pdf/route.ts (GET - 生成 PDF)
8. src/components/quotations/quotation-form.tsx
9. src/components/quotations/quotation-items-editor.tsx (含实时金额计算)
10. src/app/(dashboard)/quotations/page.tsx
11. src/app/(dashboard)/quotations/[id]/page.tsx

【mark-priced API 完整代码】

```
import { NextRequest, NextResponse } from "next/server";
import { createClient } from "@lib/supabase/server";
import { logAction } from "@lib/audit";

export async function POST(
  request: NextRequest,
  { params }: { params: { id: string } }
) {
  const supabase = createClient();

  // 校验所有 items 的 unit_price > 0
  const { data: items } = await supabase
    .from("quotation_items")
    .select("id, unit_price")
    .eq("quotation_id", params.id);

  const unpriced = items?.filter(i => !i.unit_price || i.unit_price <= 0);
  if (unpriced?.length > 0) {
    return NextResponse.json({
      error: 还有 ${unpriced.length} 项未定价
    }, { status: 400 });
  }

  // 计算总金额
  const total = items?.reduce((sum, i) =>
    sum + (Number(i.quantity) * Number(i.unit_price)), 0) || 0;

  // 更新报价单
  const { error } = await supabase
```

```

.from("quotations")
.update({
pricing_status: "priced",
total_amount: total,
updated_at: new Date().toISOString()
})
.eq("id", params.id);

if (error) return NextResponse.json({ error: error.message }, { status: 500 });

await logAction({
user_id: "system",
module: "quotations",
action: "status_change",
record_id: params.id,
after_data: { pricing_status: "priced", total_amount: total },
description: "报价单标记为已定价",
});

return NextResponse.json({ message: "已标记为已定价" });
}

```

【accept API 完整代码 - 含自动转订单】

```

import { NextRequest, NextResponse } from "next/server";
import { createClient } from "@lib/supabase/server";
import { logAction } from "@lib/audit";

export async function POST(
request: NextRequest,
{ params }: { params: { id: string } }
) {
const supabase = createClient();

// 1. 查询报价单
const { data: quotation, error: qError } = await supabase
.from("quotations")
.select(`

```

```
*,
items:quotation_items(*)
`)
.eq("id", params.id)
.single();

if (qError || !quotation) {
return NextResponse.json({ error: "报价单不存在" }, { status: 404 });
}

if (quotation.status !== "sent") {
return NextResponse.json({ error: "仅已发送的报价单可以接受" }, { status: 400 });
}

// 2. 生成订单编码
const year = new Date().getFullYear();
const { count } = await supabase
.from("orders")
.select("*", { count: "exact", head: true })
.like("code", 00-${year}-%);
const orderCode = 00-${year}-${String((count ?? 0) + 1).padStart(4, "0")};

// 3. 创建订单
const deliveryDate = new Date();
deliveryDate.setDate(deliveryDate.getDate() + 60);

const { data: order, error: orderError } = await supabase
.from("orders")
.insert({
code: orderCode,
customer_id: quotation.customer_id,
quotation_id: quotation.id,
order_date: new Date().toISOString().split("T")[0],
delivery_date: deliveryDate.toISOString().split("T")[0],
currency: quotation.currency,
total_amount: quotation.total_amount,
status: "pending",
```

```
trade_terms: quotation.trade_terms,
payment_terms: quotation.payment_terms,
})
.select()
.single();

if (orderError) return NextResponse.json({ error: orderError.message }, { status: 500 });

// 4. 复制明细到 order_items
if (quotation.items?.length > 0) {
  const orderItems = quotation.items.map((item: any) => ({
    order_id: order.id,
    product_id: item.product_id,
    quantity: item.quantity,
    unit: item.unit,
    unit_price: item.unit_price,
    notes: item.notes,
  }));

  await supabase.from("order_items").insert(orderItems);
}

// 5. 更新报价单状态
await supabase
  .from("quotations")
  .update({
    status: "accepted",
    accepted_at: new Date().toISOString(),
    updated_at: new Date().toISOString()
  })
  .eq("id", params.id);

// 6. 更新询盘状态
if (quotation.inquiry_id) {
  await supabase
    .from("inquiries")
    .update({ status: "closed", updated_at: new Date().toISOString() })
```



```
.eq("id", quotation.inquiry_id);
}

// 7. 写审计日志
await logAction({
  user_id: "system",
  module: "quotations",
  action: "status_change",
  record_id: params.id,
  record_code: quotation.code,
  after_data: { status: "accepted", order_id: order.id },
  description: 报价单 ${quotation.code} 被接受, 已转为订单 ${order.code},
});

return NextResponse.json({
  data: order,
  message: "已接受并转为订单"
});
}
```

【实时金额计算 - 在表单组件中】

```
// 在 quotation-form.tsx 中
const items = form.watch("items");
const totalAmount = useMemo(() => {
  return items.reduce((sum, item) => {
    return sum + (Number(item.quantity) || 0) * (Number(item.unit_price) || 0);
  }, 0);
}, [items]);

// 在表单底部显示
<div className="text-right text-lg font-bold">
  总金额: {formatCurrency(totalAmount, currency)}
</div>
```

请生成所有文件的完整代码，不要省略任何部分。

第十章 订单管理模块（状态机核心）

订单管理是整个

ERP

系统的"心脏"——它串起了库存、采购、发货、财务所有模块。本章重点讲订单状态机的完整实现。

10.1 订单状态机完整设计

订单的 6 个状态及流转关系：

状态	中文名	允许流转到	触发的副作用
pending	待处理	confirmed, cancelled	确认时：冻结库存 + 检查库存生成请购 + 生成应收账款
confirmed	已确认	producing, cancelled	进入生产阶段（无副作用）
producing	生产中	shipped, cancelled	发货时：扣减冻结+实际库存 + 生成出库单
shipped	已发货	completed	完成时：归档订单
completed	已完成	（终态）	归档
cancelled	已取消	（终态）	释放冻结库存 + 取消关联请购 + 取消应收

10.2 PostgreSQL 事务实现

由于状态变更涉及多个表的修改，必须用事务保证原子性。按 **Ctrl+I** 唤起 Builder，输入：

创建 sql/006_order_transition.sql, 定义 PostgreSQL 函数 transition_order:

```
CREATE OR REPLACE FUNCTION transition_order(
  p_order_id uuid,
  p_from_status text,
  p_to_status text,
  p_user_id uuid,
  p_reason text
)
RETURNS void AS $$
DECLARE
  v_order orders%ROWTYPE;
  v_item record;
  v_inv inventory%ROWTYPE;
  v_pr_code text;
  v_pr_id uuid;
  v_fin_code text;
  v_rate numeric;
  v_count int;
BEGIN
  -- 1. 查询订单并校验当前状态
  SELECT * INTO v_order FROM orders WHERE id = p_order_id FOR UPDATE;
  IF v_order.status != p_from_status THEN
    RAISE EXCEPTION '订单状态不匹配: 期望 %, 实际 %', p_from_status, v_order.status;
  END IF;

  -- 2. 校验流转合法性
  IF NOT (
    (p_from_status = 'pending' AND p_to_status IN ('confirmed', 'cancelled')) OR
    (p_from_status = 'confirmed' AND p_to_status IN ('producing', 'cancelled')) OR
    (p_from_status = 'producing' AND p_to_status IN ('shipped', 'cancelled')) OR
    (p_from_status = 'shipped' AND p_to_status = 'completed')
  ) THEN
    RAISE EXCEPTION '不允许从 % 流转到 %', p_from_status, p_to_status;
  END IF;

  -- 3. 执行状态变更的副作用
  IF p_to_status = 'confirmed' THEN
```

```
-- 3.1 冻结库存
FOR v_item IN SELECT product_id, quantity FROM order_items WHERE order_id = p_order_id LOOP
SELECT * INTO v_inv FROM inventory
WHERE product_id = v_item.product_id
AND warehouse_id = (SELECT id FROM warehouses WHERE code = 'WH001')
FOR UPDATE;
IF v_inv.id IS NULL THEN
INSERT INTO inventory (product_id, warehouse_id, quantity, frozen_quantity)
VALUES (v_item.product_id, (SELECT id FROM warehouses WHERE code = 'WH001'), 0, v_item.quantity);
ELSE
UPDATE inventory SET frozen_quantity = frozen_quantity + v_item.quantity,
updated_at = now() WHERE id = v_inv.id;
END IF;
INSERT INTO stock_movements (movement_type, product_id, warehouse_id, quantity,
reference_type, reference_id) VALUES ('freeze', v_item.product_id,
v_inv.warehouse_id, v_item.quantity, 'order', p_order_id);
END LOOP;

-- 3.2 生成应收账款
SELECT rate_to_cny INTO v_rate FROM exchange_rates
WHERE currency = v_order.currency
ORDER BY effective_date DESC LIMIT 1;

SELECT count(*) INTO v_count FROM receivables
WHERE code LIKE 'FIN-' || EXTRACT(YEAR FROM now()) || '-%';
v_fin_code := 'FIN-' || EXTRACT(YEAR FROM now()) || '-' || lpad((v_count + 1)::text, 4, '0');

INSERT INTO receivables (code, customer_id, order_id, category, amount,
currency, exchange_rate, due_date, status)
VALUES (v_fin_code, v_order.customer_id, p_order_id, 'order_payment',
v_order.total_amount, v_order.currency, COALESCE(v_rate, 1),
CURRENT_DATE + 30, 'pending');

-- 3.3 更新订单确认时间
UPDATE orders SET status = 'confirmed', confirmed_at = now(), updated_at = now()
WHERE id = p_order_id;

ELSIF p_to_status = 'cancelled' THEN
```

```

-- 3.4 取消：释放冻结库存
FOR v_item IN SELECT product_id, quantity FROM order_items WHERE order_id = p_order_id LOOP
UPDATE inventory SET frozen_quantity = GREATEST(0, frozen_quantity - v_item.quantity),
updated_at = now() WHERE product_id = v_item.product_id;
INSERT INTO stock_movements (movement_type, product_id, quantity,
reference_type, reference_id) VALUES ('unfreeze', v_item.product_id,
v_item.quantity, 'order', p_order_id);
END LOOP;
-- 3.5 取消关联请购
UPDATE purchase_requests SET status = 'rejected' WHERE order_id = p_order_id AND status = 'pending';
-- 3.6 取消关联应收
UPDATE receivables SET status = 'cancelled' WHERE order_id = p_order_id AND status = 'pending';
UPDATE orders SET status = 'cancelled', updated_at = now() WHERE id = p_order_id;

ELSIF p_to_status = 'shipped' THEN
-- 3.7 发货：扣减库存
FOR v_item IN SELECT product_id, quantity FROM order_items WHERE order_id = p_order_id LOOP
UPDATE inventory SET quantity = quantity - v_item.quantity,
frozen_quantity = GREATEST(0, frozen_quantity - v_item.quantity),
updated_at = now() WHERE product_id = v_item.product_id;
INSERT INTO stock_movements (movement_type, product_id, quantity,
reference_type, reference_id) VALUES ('outbound_sale', v_item.product_id,
-v_item.quantity, 'order', p_order_id);
END LOOP;
UPDATE orders SET status = 'shipped', shipped_at = now(), updated_at = now()
WHERE id = p_order_id;

ELSIF p_to_status = 'completed' THEN
UPDATE orders SET status = 'completed', completed_at = now(), updated_at = now()
WHERE id = p_order_id;
END IF;

-- 4. 写审计日志
INSERT INTO audit_logs (user_id, module, action, record_id, record_code,
before_data, after_data, description)
VALUES (p_user_id, 'orders', 'status_change', p_order_id, v_order.code,
jsonb_build_object('status', p_from_status),
jsonb_build_object('status', p_to_status, 'reason', p_reason),

```

```

'订单状态变更: ' || p_from_status || ' → ' || p_to_status);
END;
$$ LANGUAGE plpgsql;

```

然后在 `src/lib/order-state-machine.ts` 中用 `supabase.rpc` 调用此函数：

```

import { createClient } from "@lib/supabase/server";

type OrderStatus = "pending" | "confirmed" | "producing" | "shipped" | "completed" | "cancelled";

const TRANSITIONS: Record<OrderStatus, OrderStatus[]> = {
  pending: ["confirmed", "cancelled"],
  confirmed: ["producing", "cancelled"],
  producing: ["shipped", "cancelled"],
  shipped: ["completed"],
  completed: [],
  cancelled: []
};

export function canTransition(from: OrderStatus, to: OrderStatus): boolean {

```

```

return TRANSITIONS[from]?.includes(to) ?? false;
}

export async function transitionOrder(
  orderId: string,
  from: OrderStatus,
  to: OrderStatus,
  userId: string,
  reason?: string
): Promise<void> {
  if (!canTransition(from, to)) {
    throw new Error(不允许从 ${from} 流转到 ${to});
  }

  const supabase = createClient();
  const { error } = await supabase.rpc("transition_order", {
    p_order_id: orderId,
    p_from_status: from,
    p_to_status: to,
    p_user_id: userId,
    p_reason: reason || null,
  });

  if (error) throw new Error(状态变更失败: ${error.message});
}

```

同时创建 API: src/app/api/orders/[id]/status/route.ts
 POST 接收 { status, reason }, 调用 transitionOrder

【API 完整代码】

```

import { NextRequest, NextResponse } from "next/server";
import { createClient } from "@lib/supabase/server";
import { transitionOrder } from "@lib/order-state-machine";

export async function POST(
  request: NextRequest,
  { params }: { params: { id: string } }
) {

```

```

const body = await request.json();
const { status: toStatus, reason } = body;

const supabase = createClient();

// 查询当前状态
const { data: order } = await supabase
  .from("orders")
  .select("status, code")
  .eq("id", params.id)
  .single();

if (!order) {
  return NextResponse.json({ error: "订单不存在" }, { status: 404 });
}

try {
  await transitionOrder(params.id, order.status, toStatus, "system", reason);
  return NextResponse.json({ message: "状态变更成功" });
} catch (error: any) {
  return NextResponse.json({ error: error.message }, { status: 400 });
}
}

```

10.3 状态时间轴组件

按 **Ctrl+I** 唤起 Builder，输入：

```

创建 src/components/orders/order-status-timeline.tsx 组件：
- Props: orderId
- 调用 GET /api/orders/[id]/timeline 获取该订单的所有状态变更记录（从 audit_logs 查询）
- 用垂直时间轴展示：每个节点显示状态名 + 时间 + 操作人
- 5 个固定节点（pending/confirmed/producing/shipped/completed），已完成的节点显示绿色对勾，未到达的灰色
- 当前状态节点显示蓝色脉动效果
- 横向布局（桌面端）和纵向布局（移动端）切换

```

【完整代码】

```

"use client";

import { useQuery } from "@tanstack/react-query";
import { Check, Clock, Circle } from "lucide-react";
import { cn } from "@lib/utills";

```

```

const STATUS_FLOW = [
  { key: "pending", label: "待处理" },
  { key: "confirmed", label: "已确认" },
  { key: "producing", label: "生产中" },
  { key: "shipped", label: "已发货" },
  { key: "completed", label: "已完成" },
];

export function OrderStatusTimeline({ orderId, currentStatus }: { orderId: string; currentStatus: string }) {
  const { data: timeline } = useQuery({
    queryKey: ["order-timeline", orderId],
    queryFn: async () => {
      const res = await fetch(`/api/orders/${orderId}/timeline`);
      return res.json();
    },
  });

  const currentIndex = STATUS_FLOW.findIndex(s => s.key === currentStatus);

  return (
    <div className="flex items-center justify-between w-full">
      {STATUS_FLOW.map((status, index) => {
        const isCompleted = index < currentIndex;
        const isCurrent = index === currentIndex;
        const isFuture = index > currentIndex;

        const event = timeline?.data?.find((e: any) =>
          e.after_data?.status === status.key
        );

        return (
          <div key={status.key} className="flex items-center flex-1">
            <div className="flex flex-col items-center">
              <div className={cn(
                "w-10 h-10 rounded-full flex items-center justify-center border-2",
                isCompleted && "bg-green-500 border-green-500 text-white",

```



```

isCurrent && "bg-blue-500 border-blue-500 text-white animate-pulse",
isFuture && "bg-gray-100 border-gray-300 text-gray-400"
}}>
{isCompleted ? <Check size={18} /> :
isCurrent ? <Clock size={18} /> :
<Circle size={18} />}
</div>
<div className="mt-2 text-center">
<div className={cn(
"text-sm font-medium",
isFuture ? "text-gray-400" : "text-gray-900"
)}>
{status.label}
</div>
{event && (
<div className="text-xs text-gray-500 mt-1">
{new Date(event.created_at).toLocaleDateString()}
</div>
)}
</div>
</div>
{index < STATUS_FLOW.length - 1 && (
<div className={cn(
"flex-1 h-0.5 mx-2",
index < currentIndex ? "bg-green-500" : "bg-gray-200"
)} />
)}
</div>
);
}}
</div>
);
}

```

10.4 完整订单管理模块 Prompt

按 **Ctrl+I** 唤起 Builder，输入：

生成完整的订单管理模块。请创建以下文件：

1. src/app/api/orders/route.ts (GET 列表, POST 创建)
2. src/app/api/orders/[id]/route.ts (GET, PUT, DELETE)
3. src/app/api/orders/[id]/status/route.ts (POST - 状态变更)
4. src/app/api/orders/[id]/timeline/route.ts (GET - 状态时间轴)
5. src/components/orders/orders/order-form.tsx
6. src/components/orders/orders/order-status-timeline.tsx
7. src/components/orders/orders/order-status-button.tsx (动态显示可流转状态按钮)
8. src/app/(dashboard)/orders/page.tsx
9. src/app/(dashboard)/orders/[id]/page.tsx (含状态时间轴 + 关联单据)

【订单列表页表格列】

订单号、客户、下单日期、交货日期、币种、总金额(彩色)、状态(彩色badge)、明细数、操作

【详情页布局】

- 顶部: 返回按钮 + 订单号 + 状态badge + 操作按钮组
- 操作按钮组 (根据当前状态动态显示) :
- pending: [确认订单] [取消订单]
- confirmed: [开始生产] [取消订单]
- producing: [标记发货] [取消订单]
- shipped: [标记完成]
- completed: [查看]
- 状态变更需二次确认弹窗
- 中部: 订单信息卡 (左右两列) + OrderStatusTimeline
- 下部: Tab 切换 明细/请购单/采购订单/发货单/应收账款

【状态变更确认弹窗逻辑】

```
const messages = {
  confirmed: "确认订单将冻结库存并生成应收账款，是否继续？",
  producing: "确认开始生产？",
  shipped: "标记发货将扣减库存，是否继续？",
  completed: "确认完成此订单？",
  cancelled: "取消订单将释放冻结库存并取消关联的请购和应收，是否继续？",
};
```

【order-status-button.tsx 完整代码】

```
"use client";

import { useState } from "react";
import { Button } from "@/components/ui/button";
import { ConfirmDialog } from "@/components/common/confirm-dialog";
import { toast } from "sonner";
import { useMutation, useQueryClient } from "@tanstack/react-query";

const STATUS_OPTIONS: Record<string, { label: string; variant: "default" | "destructive" }>[] = {
  pending: [
    { label: "确认订单", variant: "default" },
    { label: "取消订单", variant: "destructive" },
  ],

```

```
confirmed: [
  { label: "开始生产", variant: "default" },
  { label: "取消订单", variant: "destructive" },
],
producing: [
  { label: "标记发货", variant: "default" },
  { label: "取消订单", variant: "destructive" },
],
shipped: [{ label: "标记完成", variant: "default" }],
completed: [],
cancelled: [],
};

const STATUS_MAP: Record<string, string> = {
  "确认订单": "confirmed",
  "开始生产": "producing",
  "标记发货": "shipped",
  "标记完成": "completed",
  "取消订单": "cancelled",
};

const CONFIRM_MESSAGES: Record<string, string> = {
  confirmed: "确认订单将冻结库存并生成应收账款，是否继续？",
  producing: "确认开始生产？",
  shipped: "标记发货将扣减库存，是否继续？",
  completed: "确认完成此订单？",
  cancelled: "取消订单将释放冻结库存并取消关联的请购和应收，是否继续？",
};

export function OrderStatusButton({ orderId, currentStatus }: { orderId: string; currentStatus:
string }) {
  const [confirmOpen, setConfirmOpen] = useState(false);
  const [targetStatus, setTargetStatus] = useState("");
  const queryClient = useQueryClient();

  const mutation = useMutation({
```

```
mutationFn: async (status: string) => {
  const res = await fetch(`/api/orders/${orderId}/status`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ status }),
  });
  if (!res.ok) throw new Error("状态变更失败");
  return res.json();
},
onSuccess: () => {
  toast.success("状态变更成功");
  queryClient.invalidateQueries({ queryKey: ["order", orderId] });
  queryClient.invalidateQueries({ queryKey: ["orders"] });
},
onError: (err: any) => toast.error(err.message),
});

const options = STATUS_OPTIONS[currentStatus] || [];

const handleClick = (label: string) => {
  const status = STATUS_MAP[label];
  setTargetStatus(status);
  setConfirmOpen(true);
};

return (
  <>
  <div className="flex gap-2">
    {options.map((opt) => (
      <Button
        key={opt.label}
        variant={opt.variant}
        onClick={() => handleClick(opt.label)}
      >
        {opt.label}
      </Button>
    ))}
  </div>
)
```

```
</Button>

)}}
</div>

<ConfirmDialog
open={confirmOpen}
onOpenChange={setConfirmOpen}
title="订单状态变更"
description={CONFIRM_MESSAGES[targetStatus]}
variant={targetStatus === "cancelled" ? "destructive" : "default"}
onConfirm={() => {
mutation.mutate(targetStatus);
setConfirmOpen(false);
}}
/>
</>
);
}
```

请生成所有文件的完整代码。

本章关键点：订单状态机是 ERP 的核心。三个关键设计：① 用 PostgreSQL 存储过程保证事务原子性；② 每个状态变更触发明确的副作用（库存/财务/采购）；③ 不可逆操作必须二次确认。

第十一章 采购管理模块

采购管理解决"订单需要哪些货、找谁买、什么时候到"的问题。

11.1 完整模块生成 Prompt

按 **Ctrl+I** 唤起 Builder，输入：

生成完整的采购管理模块，包含请购单和采购订单两部分。请创建以下文件：

1. src/app/api/purchase-requests/route.ts (GET 列表, POST 创建)
2. src/app/api/purchase-requests/[id]/route.ts (GET, PUT)
3. src/app/api/purchase-requests/[id]/approve/route.ts (POST - 审批通过)
4. src/app/api/purchase-requests/[id]/reject/route.ts (POST - 审批拒绝)
5. src/app/api/purchase-requests/[id]/convert-to-purchase-order/route.ts (POST - 转采购订单)
6. src/app/api/purchase-orders/route.ts (GET, POST)
7. src/app/api/purchase-orders/[id]/route.ts (GET, PUT, DELETE)
8. src/app/api/purchase-orders/[id]/arrive/route.ts (POST - 到货登记)
9. src/app/api/purchase-orders/[id]/warehouse/route.ts (POST - 入库)
10. src/app/api/purchase-orders/[id]/pay/route.ts (POST - 付款)
11. src/app/(dashboard)/purchases/page.tsx (双 Tab: 请购单 / 采购订单)
12. src/app/(dashboard)/purchases/[id]/page.tsx

【convert-to-purchase-order API 完整代码】

```
import { NextRequest, NextResponse } from "next/server";
import { createClient } from "@/lib/supabase/server";
import { generateFormattedCode } from "@/lib/code-generator";
import { logAction } from "@/lib/audit";

export async function POST(
  request: NextRequest,
  { params }: { params: { id: string } }
) {
  const body = await request.json();
  // body: { supplier_id, items: [{product_id, quantity, unit_price}] }

  const supabase = createClient();

  // 1. 查询请购单
  const { data: pr } = await supabase
    .from("purchase_requests")
    .select("*")
    .eq("id", params.id)
    .single();

  if (!pr || pr.status !== "approved") {
    return NextResponse.json({ error: "请购单不存在或未审核" }, { status: 400 });
  }

  // 2. 生成采购订单编码
  const code = await generateFormattedCode("purchase_orders", "PO");

  // 3. 计算总金额
  const totalAmount = body.items.reduce((sum: number, item: any) =>
```

```
sum + item.quantity * item.unit_price, 0);

// 4. 创建采购订单
const { data: po, error } = await supabase
  .from("purchase_orders")
  .insert({
    code,
    supplier_id: body.supplier_id,
    purchase_request_id: params.id,
    order_date: new Date().toISOString().split("T")[0],
    currency: "CNY",
    total_amount: totalAmount,
    status: "pending",
    trade_terms: "EXW",
    payment_terms: "T/T",
  })
  .select()
  .single();

if (error) return NextResponse.json({ error: error.message }, { status: 500 });

// 5. 创建采购明细
const items = body.items.map((item: any) => {
  purchase_order_id: po.id,
  product_id: item.product_id,
  quantity: item.quantity,
  unit_price: item.unit_price,
});

await supabase.from("purchase_order_items").insert(items);

// 6. 更新请购单状态
await supabase
  .from("purchase_requests")
  .update({ status: "converted", updated_at: new Date().toISOString() })
  .eq("id", params.id);

await logAction({
```

```

user_id: "system",
module: "purchase_orders",
action: "create",
record_id: po.id,
record_code: po.code,
after_data: po,
description: 从请购单 ${pr.code} 转为采购订单 ${po.code},
});

return NextResponse.json({ data: po, message: "已转为采购订单" });
}

```

【到货登记 API - 分批到货】

```

export async function POST(
  request: NextRequest,
  { params }: { params: { id: string } }
) {
  const body = await request.json();
  // body: { items: [{id, arrived_quantity}] }

  const supabase = createClient();

  // 1. 查询采购订单状态
  const { data: po } = await supabase
    .from("purchase_orders")
    .select("status")
    .eq("id", params.id)
    .single();

  if (!["sent", "partial_arrived"].includes(po.status)) {
    return NextResponse.json({ error: "当前状态不允许到货登记" }, { status: 400 });
  }

  // 2. 更新每个明细的已到货数量
  let allArrived = true;
  for (const item of body.items) {
    if (item.arrived_quantity <= 0) continue;

```



```

const { data: poi } = await supabase
  .from("purchase_order_items")
  .select("arrived_quantity, quantity")
  .eq("id", item.id)
  .single();

const newArrived = (poi.arrived_quantity || 0) + item.arrived_quantity;
await supabase
  .from("purchase_order_items")
  .update({ arrived_quantity: newArrived })
  .eq("id", item.id);

if (newArrived < poi.quantity) allArrived = false;
}

// 3. 更新采购订单状态
await supabase
  .from("purchase_orders")
  .update({
    status: allArrived ? "arrived" : "partial_arrived",
    updated_at: new Date().toISOString()
  })
  .eq("id", params.id);

return NextResponse.json({ message: "到货登记成功" });
}

```

【入库 API - 触发库存增加】

```

export async function POST(
  request: NextRequest,
  { params }: { params: { id: string } }
) {
  const body = await request.json();
  // body: { items: [{id, warehoused_quantity}], warehouse_id }

  const supabase = createClient();

  // 1. 查询采购订单和明细

```

```
const { data: po } = await supabase
  .from("purchase_orders")
  .select(`
    *,
    items:purchase_order_items(*)
  `)
  .eq("id", params.id)
  .single();

if (!["arrived", "partial_arrived"].includes(po.status)) {
  return NextResponse.json({ error: "当前状态不允许入库" }, { status: 400 });
}

// 2. 对每个入库项更新库存
for (const item of body.items) {
  if (item.warehoused_quantity <= 0) continue;

  const poi = po.items.find((i: any) => i.id === item.id);
  if (!poi) continue;

  // 更新 warehoused_quantity
  await supabase
    .from("purchase_order_items")
    .update({
      warehoused_quantity: (poi.warehoused_quantity || 0) + item.warehoused_quantity
    })
    .eq("id", item.id);

  // 增加库存
  const { data: inv } = await supabase
    .from("inventory")
    .select("id, quantity")
    .eq("product_id", poi.product_id)
    .eq("warehouse_id", body.warehouse_id)
    .single();

  if (inv) {
    await supabase
```

```
.from("inventory")
.update({
  quantity: inv.quantity + item.warehoused_quantity,
  updated_at: new Date().toISOString()
})
.eq("id", inv.id);
} else {
  await supabase
    .from("inventory")
    .insert({
      product_id: poi.product_id,
      warehouse_id: body.warehouse_id,
      quantity: item.warehoused_quantity,
    });
}

// 生成 stock_movement
await supabase.from("stock_movements").insert({
  movement_type: "inbound_purchase",
  product_id: poi.product_id,
  warehouse_id: body.warehouse_id,
  quantity: item.warehoused_quantity,
  reference_type: "purchase_order",
  reference_id: params.id,
});
}

// 3. 检查是否全部入库
const allWarehoused = po.items.every((item: any) =>
  (item.warehoused_quantity || 0) +
  (body.items.find((b: any) => b.id === item.id)?.warehoused_quantity || 0)
  >= item.quantity
);

if (allWarehoused) {
```

```

await supabase
.from("purchase_orders")
.update({ status: "warehoused", updated_at: new Date().toISOString() })
.eq("id", params.id);
}

return NextResponse.json({ message: "入库成功" });
}

```

【付款 API - 生成应付账款】

```

export async function POST(
  request: NextRequest,
  { params }: { params: { id: string } }
) {
  const supabase = createClient();

  const { data: po } = await supabase
    .from("purchase_orders")
    .select("*")
    .eq("id", params.id)
    .single();

  if (po.status !== "warehoused") {
    return NextResponse.json({ error: "仅已入库的采购订单可以付款" }, { status: 400 });
  }

  // 生成应付账款

  const year = new Date().getFullYear();
  const { count } = await supabase
    .from("payables")
    .select("*", { count: "exact", head: true })
    .like("code", PAY-`${year}`-%);

  const payCode = PAY-`${year}`-`${String((count ?? 0) + 1).padStart(4, "0")}`;

  const dueDate = new Date();
  dueDate.setDate(dueDate.getDate() + 30);

  await supabase.from("payables").insert({

```

```
code: payCode,
supplier_id: po.supplier_id,
purchase_order_id: params.id,
category: "purchase_payment",
amount: po.total_amount,
currency: po.currency,
due_date: dueDate.toISOString().split("T")[0],
status: "pending",
});

// 更新采购订单状态
await supabase
  .from("purchase_orders")
  .update({ status: "paid", updated_at: new Date().toISOString() })
  .eq("id", params.id);

return NextResponse.json({ message: "已生成应付账款" });
}
```

请生成所有文件的完整代码。

第十二章 库存管理模块

库存管理采用三 Tab 设计：库存查询、入库管理、出库管理。

12.1 完整模块生成 Prompt

按 **Ctrl+I** 唤起 Builder，输入：

创建 `src/app/(dashboard)/inventory/page.tsx`，包含 3 个 Tab：

Tab 1: 库存查询

- 调用 `GET /api/inventory` 获取数据（join products 表显示产品信息）
- 表格列：产品编码、产品名称、分类、库存数量、冻结数量、可用数量、仓库、库位、操作
- 可用数量 = 库存 - 冻结，用绿色显示（ ≥ 0 ），红色显示（ < 0 ）
- 行操作：[冻结] [解冻] [调整] [查看流水]
- 冻结/解冻按钮：弹出输入数量的小弹窗
- 调整按钮：弹出盘盈/盘亏表单
- 查看流水按钮：跳转到该产品的 `stock_movements` 列表

Tab 2: 入库管理

- 调用 `GET /api/stock-movements?movement_type=inbound`
- 表格列：编号、来源类型、产品、数量、仓库、关联单据、操作人、时间
- 来源类型：采购入库/退货入库/盘盈

Tab 3: 出库管理

- 类似入库，来源类型：销售出库/退货出库/盘亏

同时创建：

- `src/app/api/inventory/route.ts` (GET)
- `src/app/api/stock-movements/route.ts` (GET 列表，支持 `movement_type` 筛选)
- `src/app/api/inventory/freeze/route.ts` (POST)
- `src/app/api/inventory/unfreeze/route.ts` (POST)
- `src/app/api/inventory/adjust/route.ts` (POST - 盘盈/盘亏)
- `src/app/api/inventory/inbound/route.ts` (POST - 手动入库)
- `src/app/api/inventory/outbound/route.ts` (POST - 手动出库)

【库存查询 API 完整代码】

```
import { NextRequest, NextResponse } from "next/server";
import { createClient } from "@/lib/supabase/server";

export async function GET(request: NextRequest) {
  const searchParams = request.nextUrl.searchParams;
  const warehouseId = searchParams.get("warehouse_id");
  const search = searchParams.get("search");

  const supabase = createClient();
  let query = supabase
    .from("inventory")
    .select(`
      *,
      product:products(id, code, name, category:product_categories(name)),
      warehouse:warehouses(name)
    `);

  if (warehouseId) query = query.eq("warehouse_id", warehouseId);
  if (search) {
    query = query.ilike("product.name", `%${search}%`);
  }
}
```

```
const { data, error } = await query;

if (error) return NextResponse.json({ error: error.message }, { status: 500 });

return NextResponse.json({ data: data ?? [] });

}
```

【库存调整 API - 盘盈/盘亏 完整代码】

```
import { NextRequest, NextResponse } from "next/server";
import { createClient } from "@lib/supabase/server";
import { logAction } from "@lib/audit";

export async function POST(request: NextRequest) {
  const body = await request.json();
  // body: { product_id, warehouse_id, actual_quantity, reason }

  const supabase = createClient();

  // 1. 查询当前库存
  const { data: inv } = await supabase
    .from("inventory")
    .select("id, quantity")
    .eq("product_id", body.product_id)
    .eq("warehouse_id", body.warehouse_id)
    .single();

  const currentQty = inv?.quantity || 0;
  const diff = body.actual_quantity - currentQty;

  if (diff === 0) {
    return NextResponse.json({ message: "数量无变化" });
  }

  // 2. 更新库存
  if (inv) {
    await supabase
      .from("inventory")
      .update({
        quantity: body.actual_quantity,
        updated_at: new Date().toISOString()
      })
    
```

```
})
.eq("id", inv.id);
} else {
await supabase
.from("inventory")
.insert({
product_id: body.product_id,
warehouse_id: body.warehouse_id,
quantity: body.actual_quantity,
});
}

// 3. 生成 stock_movement
const movementType = diff > 0 ? "inbound_adjust_gain" : "outbound_adjust_loss";
await supabase.from("stock_movements").insert({
movement_type: movementType,
product_id: body.product_id,
warehouse_id: body.warehouse_id,
quantity: diff, // 正=盘盈, 负=盘亏
before_quantity: currentQty,
after_quantity: body.actual_quantity,
notes: body.reason,
});

// 4. 写审计日志
await logAction({
user_id: "system",
module: "inventory",
action: "update",
record_id: inv?.id,
before_data: { quantity: currentQty },
after_data: { quantity: body.actual_quantity, reason: body.reason },
description: `库存调整: ${currentQty} → ${body.actual_quantity} (${diff > 0 ? "盘盈" : "盘亏"}${Math.abs(diff)})`,
});
```



```
return NextResponse.json({ message: "调整成功" });  
}
```

请生成所有文件的完整代码。

12.2 库存操作工具函数

按 **Ctrl+I** 唤起 Builder，输入：

创建 `src/lib/inventory.ts`，实现以下函数：

1. `freezeStock(productId, warehouseId, quantity, referenceType, referenceId)`
2. `unfreezeStock(productId, warehouseId, quantity, referenceType, referenceId)`
3. `inboundStock(productId, warehouseId, quantity, movementType, referenceType, referenceId)`
4. `outboundStock(productId, warehouseId, quantity, movementType, referenceType, referenceId)`
5. `adjustStock(productId, warehouseId, actualQuantity, reason)`

所有函数用事务（多个 supabase 调用 + try-catch 回滚），并写审计日志。

【完整代码】

```
import { createClient } from "@lib/supabase/server";  
import { logAction } from "@lib/audit";  
  
export async function freezeStock(  
  productId: string, warehouseId: string, quantity: number,  
  referenceType: string, referenceId: string  
) {  
  const supabase = createClient();  
  
  const { data: inv } = await supabase  
    .from("inventory")  
    .select("id, frozen_quantity")  
    .eq("product_id", productId)  
    .eq("warehouse_id", warehouseId)  
    .single();  
  
  if (!inv) throw new Error("库存记录不存在");  
  
  await supabase  
    .from("inventory")  
    .update({  
      frozen_quantity: inv.frozen_quantity + quantity,  
      updated_at: new Date().toISOString()  
    })
```

```
})  
.eq("id", inv.id);  
  
await supabase.from("stock_movements").insert({  
  movement_type: "freeze",  
  product_id: productId,  
  warehouse_id: warehouseId,  
  quantity,  
  reference_type: referenceType,  
  reference_id: referenceId,  
});  
}  
  
export async function unfreezeStock(  
  productId: string, warehouseId: string, quantity: number,  
  referenceType: string, referenceId: string  
) {  
  const supabase = createClient();  
  
  const { data: inv } = await supabase  
    .from("inventory")  
    .select("id, frozen_quantity")  
    .eq("product_id", productId)  
    .eq("warehouse_id", warehouseId)  
    .single();  
  
  if (!inv) throw new Error("库存记录不存在");  
  
  await supabase  
    .from("inventory")  
    .update({  
      frozen_quantity: Math.max(0, inv.frozen_quantity - quantity),  
      updated_at: new Date().toISOString()  
    })  
    .eq("id", inv.id);  
  
  await supabase.from("stock_movements").insert({  
    movement_type: "unfreeze",
```

```
product_id: productId,
warehouse_id: warehouseId,
quantity,
reference_type: referenceType,
reference_id: referenceId,
});
}

export async function inboundStock(
productId: string, warehouseId: string, quantity: number,
movementType: string, referenceType: string, referenceId: string
) {
const supabase = createClient();

const { data: inv } = await supabase
.from("inventory")
.select("id, quantity")
.eq("product_id", productId)
.eq("warehouse_id", warehouseId)
.single();

if (inv) {
await supabase
.from("inventory")
.update({
quantity: inv.quantity + quantity,
updated_at: new Date().toISOString()
})
.eq("id", inv.id);
} else {
await supabase
.from("inventory")
.insert({
product_id: productId,
warehouse_id: warehouseId,
```

```
quantity,
});
}

await supabase.from("stock_movements").insert({
  movement_type: movementType,
  product_id: productId,
  warehouse_id: warehouseId,
  quantity,
  reference_type: referenceType,
  reference_id: referenceId,
});
}

export async function outboundStock(
  productId: string, warehouseId: string, quantity: number,
  movementType: string, referenceType: string, referenceId: string
) {
  const supabase = createClient();

  const { data: inv } = await supabase
    .from("inventory")
    .select("id, quantity, frozen_quantity")
    .eq("product_id", productId)
    .eq("warehouse_id", warehouseId)
    .single();

  if (!inv) throw new Error("库存记录不存在");
  if (inv.quantity < quantity) throw new Error("库存不足");

  await supabase
    .from("inventory")
    .update({
      quantity: inv.quantity - quantity,
      updated_at: new Date().toISOString()
    })
    .eq("id", inv.id);
```

```
await supabase.from("stock_movements").insert({
  movement_type: movementType,
  product_id: productId,
  warehouse_id: warehouseId,
  quantity: -quantity, // 负数表示出库
  reference_type: referenceType,
  reference_id: referenceId,
});
}
```

第十三章 发货管理模块

发货管理处理订单的物流环节，支持分批发货。

13.1 完整模块生成 Prompt

按 **Ctrl+I** 唤起 Builder，输入：

生成完整的发货管理模块。请创建以下文件：

1. src/app/api/shipments/route.ts (GET, POST)
2. src/app/api/shipments/[id]/route.ts (GET, PUT, DELETE)
3. src/app/api/shipments/[id]/ship/route.ts (POST - 标记已发货)
4. src/app/api/shipments/[id]/deliver/route.ts (POST - 标记已签收)
5. src/components/shipments/shipment-form.tsx
6. src/app/(dashboard)/shipments/page.tsx
7. src/app/(dashboard)/shipments/[id]/page.tsx

【发货表单字段】

- 选择订单（必须是 status=producing 的订单）
- 发货日期（默认今天）
- 物流方式（下拉：sea/air/express/land）
- 物流公司、物流单号
- 集装箱号、提单号（仅海运时必填）
- 报关单号（可选）

【装箱明细表】

- 自动带出订单的所有 order_items
- 每行显示：产品、订单数量、已发货数量、本次发货数量（可编辑，默认=订单数量-已发货数量）
- 每行可输入：箱数、重量(kg)、体积(cbm)
- 底部显示总箱数、总重量、总体积

【POST 创建 API 完整代码】

```
import { NextRequest, NextResponse } from "next/server";
import { createClient } from "@/lib/supabase/server";
import { generateFormattedCode } from "@/lib/code-generator";
```

```
import { logAction } from "@lib/audit";

export async function POST(request: NextRequest) {
  const body = await request.json();
  const supabase = createClient();

  // 1. 查询订单状态
  const { data: order } = await supabase
    .from("orders")
    .select("id, code, status, customer_id")
    .eq("id", body.order_id)
    .single();

  if (!order || order.status !== "producing") {
    return NextResponse.json({ error: "订单不存在或未在生产中" }, { status: 400 });
  }

  // 2. 生成发货单编码
  const code = await generateFormattedCode("shipments", "SH");

  // 3. 计算总重量和总体积
  const totalWeight = body.items.reduce((sum: number, i: any) =>
    sum + (i.weight_kg || 0), 0);
  const totalVolume = body.items.reduce((sum: number, i: any) =>
    sum + (i.volume_cbm || 0), 0);

  // 4. 创建发货单
  const { data: shipment, error } = await supabase
    .from("shipments")
    .insert({
      code,
      order_id: body.order_id,
      order_code: order.code,
      customer_id: order.customer_id,
      shipment_date: body.shipment_date,
      shipping_method: body.shipping_method,
      logistics_company: body.logistics_company,
      tracking_number: body.tracking_number,
```

```
container_number: body.container_number,
bl_number: body.bl_number,
customs_declaration_number: body.customs_declaration_number,
total_weight_kg: totalWeight,
total_volume_cbm: totalVolume,
status: "pending",
})
.select()
.single();

if (error) return NextResponse.json({ error: error.message }, { status: 500 });

// 5. 创建发货明细
if (body.items?.length > 0) {
  const items = body.items.filter((i: any) => i.quantity > 0).map((item: any) => ({
    shipment_id: shipment.id,
    order_item_id: item.order_item_id,
    product_id: item.product_id,
    quantity: item.quantity,
    cartons: item.cartons,
    weight_kg: item.weight_kg,
    volume_cbm: item.volume_cbm,
  }));

  await supabase.from("shipment_items").insert(items);
}

await logAction({
  user_id: "system",
  module: "shipments",
  action: "create",
  record_id: shipment.id,
  record_code: shipment.code,
  after_data: shipment,
  description: 创建发货单 ${shipment.code},
});
```

```
return NextResponse.json({ data: shipment, message: "创建成功" }, { status: 201 });
}
```

【ship API - 标记已发货，触发库存扣减和订单状态更新 完整代码】

```
import { NextRequest, NextResponse } from "next/server";
import { createClient } from "@lib/supabase/server";
import { logAction } from "@lib/audit";

export async function POST(
  request: NextRequest,
  { params }: { params: { id: string } }
) {
  const supabase = createClient();

  // 1. 查询发货单和明细
  const { data: shipment } = await supabase
    .from("shipments")
    .select(`
      *,
      items:shipment_items(*),
      order:orders(id, code, status)
    `)
    .eq("id", params.id)
    .single();

  if (!shipment || shipment.status !== "pending") {
    return NextResponse.json({ error: "发货单不存在或状态不允许" }, { status: 400 });
  }

  // 2. 更新发货单状态
  await supabase
    .from("shipments")
    .update({
      status: "shipped",
      shipped_at: new Date().toISOString(),
      updated_at: new Date().toISOString()
    })
    .eq("id", params.id)
    .single();

  logAction("发货单状态更新", { id: params.id, status: "shipped" });

  return NextResponse.json({ data: shipment, message: "发货单状态更新成功" }, { status: 200 });
}
```



```
})  
.eq("id", params.id);  
  
// 3. 对每个明细扣减库存并更新 order_items.shipped_quantity  
const warehouseId = (await supabase  
.from("warehouses")  
.select("id")  
.eq("code", "WH001")  
.single()).data?.id;  
  
for (const item of shipment.items) {  
  // 扣减实际库存和冻结库存  
  const { data: inv } = await supabase  
.from("inventory")  
.select("id, quantity, frozen_quantity")  
.eq("product_id", item.product_id)  
.eq("warehouse_id", warehouseId)  
.single();  
  
  if (inv) {  
    await supabase  
.from("inventory")  
.update({  
      quantity: Math.max(0, inv.quantity - item.quantity),  
      frozen_quantity: Math.max(0, inv.frozen_quantity - item.quantity),  
      updated_at: new Date().toISOString()  
    })  
.eq("id", inv.id);  
  }  
  
  // 生成 stock_movement  
  await supabase.from("stock_movements").insert({  
    movement_type: "outbound_sale",  
    product_id: item.product_id,  
    warehouse_id: warehouseId,  
    quantity: -item.quantity,
```

```
reference_type: "shipment",
reference_id: params.id,
});

// 更新 order_items.shipped_quantity
if (item.order_item_id) {
  const { data: oi } = await supabase
    .from("order_items")
    .select("shipped_quantity")
    .eq("id", item.order_item_id)
    .single();

  await supabase
    .from("order_items")
    .update({
      shipped_quantity: (oi?.shipped_quantity || 0) + item.quantity
    })
    .eq("id", item.order_item_id);
}

// 4. 检查所有 order_items 是否已全部发货
const { data: orderItems } = await supabase
  .from("order_items")
  .select("quantity, shipped_quantity")
  .eq("order_id", shipment.order_id);

const allShipped = orderItems?.every(i =>
  (i.shipped_quantity || 0) >= i.quantity);

if (allShipped) {
  // 更新订单状态为 shipped
  await supabase
    .from("orders")
    .update({
      status: "shipped",
      shipped_at: new Date().toISOString(),
    })
```

```
updated_at: new Date().toISOString()
})
.eq("id", shipment.order_id);
}

// 5. 自动生成单证草稿
await supabase.from("documents").insert([
{
code: DOC-`${Date.now()}-1`,
order_id: shipment.order_id,
shipment_id: params.id,
document_type: "commercial_invoice",
status: "draft",
},
{
code: DOC-`${Date.now()}-2`,
order_id: shipment.order_id,
shipment_id: params.id,
document_type: "packing_list",
status: "draft",
},
]);

await logAction({
user_id: "system",
module: "shipments",
action: "status_change",
record_id: params.id,
record_code: shipment.code,
after_data: { status: "shipped" },
description: 发货单 ${shipment.code} 已发货,
});

return NextResponse.json({ message: "已标记发货" });
}
```

请生成所有文件的完整代码。

第十四章 财务管理模块

财务管理分为应收和应付两个 Tab。

14.1 完整模块生成 Prompt

按 **Ctrl+I** 唤起 Builder，输入：

生成完整的财务管理模块。请创建以下文件：

1. src/app/api/receivables/route.ts (GET, POST)
2. src/app/api/receivables/[id]/route.ts (GET, PUT, DELETE)
3. src/app/api/receivables/[id]/payment/route.ts (POST - 收款登记)
4. src/app/api/payables/route.ts (GET, POST)
5. src/app/api/payables/[id]/route.ts (GET, PUT, DELETE)
6. src/app/api/payables/[id]/payment/route.ts (POST - 付款登记)
7. src/app/api/finance/summary/route.ts (GET - 财务看板)
8. src/app/api/finance/overdue-count/route.ts (GET - 逾期统计)
9. src/app/(dashboard)/finance/page.tsx (含顶部看板 + 双 Tab)
10. src/components/finance/payment-dialog.tsx (收款/付款弹窗)

【财务看板 API 完整代码】

```
import { NextResponse } from "next/server";

import { createClient } from "@/lib/supabase/server";

export async function GET() {
  const supabase = createClient();

  // 应收总额

  const { data: receivables } = await supabase
    .from("receivables")
    .select("amount, received_amount, status")
    .is("deleted_at", null);

  const totalReceivable = receivables?.reduce((sum, r) =>
    sum + Number(r.amount), 0) || 0;

  const totalReceived = receivables?.reduce((sum, r) =>
    sum + Number(r.received_amount || 0), 0) || 0;

  const overdueReceivable = receivables
    ?.filter(r => r.status === "overdue")
    .reduce((sum, r) => sum + Number(r.amount), 0) || 0;
```

```
// 应付总额

const { data: payables } = await supabase
  .from("payables")
  .select("amount, paid_amount, status")
  .is("deleted_at", null);

const totalPayable = payables?.reduce((sum, p) =>
  sum + Number(p.amount), 0) || 0;
const totalPaid = payables?.reduce((sum, p) =>
  sum + Number(p.paid_amount || 0), 0) || 0;
const overduePayable = payables
  ?.filter(p => p.status === "overdue")
  .reduce((sum, p) => sum + Number(p.amount), 0) || 0;

return NextResponse.json({
  receivable: {
    total: totalReceivable,
    received: totalReceived,
    overdue: overdueReceivable,
    pending: totalReceivable - totalReceived,
  },
  payable: {
    total: totalPayable,
    paid: totalPaid,
    overdue: overduePayable,
    pending: totalPayable - totalPaid,
  },
});
}
```

【收款登记 API - 支持部分收款 完整代码】

```
import { NextRequest, NextResponse } from "next/server";
import { createClient } from "@lib/supabase/server";
import { logAction } from "@lib/audit";
```

```
export async function POST(
  request: NextRequest,
  { params }: { params: { id: string } }
) {
  const body = await request.json();
  // body: { amount, payment_date, payment_method, bank_reference, notes }

  const supabase = createClient();

  // 1. 查询应收
  const { data: rec } = await supabase
    .from("receivables")
    .select("*")
    .eq("id", params.id)
    .single();

  if (!rec) return NextResponse.json({ error: "应收不存在" }, { status: 404 });
  if (["received", "cancelled"].includes(rec.status)) {
    return NextResponse.json({ error: "当前状态不允许收款" }, { status: 400 });
  }

  // 2. 创建收款记录
  await supabase.from("receivable_payments").insert({
    receivable_id: params.id,
    amount: body.amount,
    payment_date: body.payment_date,
    payment_method: body.payment_method,
    bank_reference: body.bank_reference,
    exchange_rate: rec.exchange_rate,
    notes: body.notes,
  });

  // 3. 更新应收的 received_amount 和 status
  const newReceived = Number(rec.received_amount || 0) + Number(body.amount);
  let newStatus;
  if (newReceived >= Number(rec.amount)) {
    newStatus = "received";
  }
```

```

} else if (newReceived > 0) {
  newStatus = "partial";
} else {
  newStatus = "pending";
}

await supabase
  .from("receivables")
  .update({
    received_amount: newReceived,
    status: newStatus,
    received_at: newStatus === "received" ? new Date().toISOString() : null,
    updated_at: new Date().toISOString()
  })
  .eq("id", params.id);

await logAction({
  user_id: "system",
  module: "receivables",
  action: "update",
  record_id: params.id,
  record_code: rec.code,
  after_data: { received_amount: newReceived, status: newStatus },
  description: 收款 ${body.amount}, 累计已收 ${newReceived}/${rec.amount},
});

return NextResponse.json({ message: "收款成功" });
}

```

【逾期检测 - Supabase 定时任务】

在 Supabase Dashboard → Database → Functions 创建 pg_cron 任务：

```

SELECT cron.schedule(
  'check-overdue-receivables',
  '0 0 *', -- 每天 0 点
  $$
UPDATE receivables

```

```

SET status = 'overdue'

WHERE due_date < CURRENT_DATE

AND status IN ('pending', 'partial')

AND deleted_at IS NULL;

$$

);

SELECT cron.schedule(
'check-overdue-payables',
'0 0 *',
$$

UPDATE payables
SET status = 'overdue'
WHERE due_date < CURRENT_DATE
AND status IN ('pending', 'partial')
AND deleted_at IS NULL;

$$

);

```

【汇率换算工具 - src/lib/exchange-rate.ts】

```

import { createClient } from "@lib/supabase/server";

export async function getRate(currency: string): Promise<number> {
  if (currency === "CNY") return 1;
  const supabase = createClient();
  const { data } = await supabase
    .from("exchange_rates")
    .select("rate_to_cny")
    .eq("currency", currency)
    .order("effective_date", { ascending: false })
    .limit(1)
    .single();
  return data?.rate_to_cny || 1;
}

```



```
export function convertToCNY(amount: number, currency: string, rate: number): number {
  if (currency === "CNY") return amount;
  return amount * rate;
}

export function formatMultiCurrency(amount: number, currency: string, rate: number) {
  const original = formatCurrency(amount, currency);
  if (currency === "CNY") return original;
  const cnr = convertToCNY(amount, currency, rate);
  return `${original} (~ ${formatCurrency(cnr, "CNY")})`;
}
```

请生成所有文件的完整代码。

第十五章 单证管理模块

单证管理是外贸 ERP 区别于普通进销存的关键模块。

15.1 单证类型

类型代码	中文名	说明
commercial_invoice	商业发票	结算和报关的核心单据
packing_list	装箱单	列明每箱内容
bill_of_lading	提单	海运物权凭证
certificate_of_origin	原产地证	证明货物原产国
customs_declaration	报关单	海关申报单据
inspection_certificate	商检证	商品检验证明
insurance_policy	保险单	货物运输保险
other	其他	其他类型单证

15.2 完整模块生成 Prompt

按 Ctrl+I 唤起 Builder，输入：

生成完整的单证管理模块。请创建以下文件：

1. src/app/api/documents/route.ts (GET, POST)
2. src/app/api/documents/[id]/route.ts (GET, PUT, DELETE)
3. src/app/api/documents/upload/route.ts (POST - 上传文件到 Storage)
4. src/app/api/documents/[id]/generate-pdf/route.ts (POST - 自动生成 PDF)
5. src/app/(dashboard)/documents/page.tsx
6. src/app/(dashboard)/documents/[id]/page.tsx

【Storage 配置】

在 Supabase Dashboard → Storage → New bucket, 名称填"documents", 勾选"Public bucket"

【生成 Commercial Invoice PDF 完整代码】

```
import { NextRequest, NextResponse } from "next/server";
import { createClient } from "@lib/supabase/server";
import { createAdminClient } from "@lib/supabase/admin";
import jsPDF from "jspdf";
import autoTable from "jspdf-autotable";

export async function POST(
  request: NextRequest,
  { params }: { params: { id: string } }
) {
  const supabase = createClient();

  // 1. 查询单证和关联订单
  const { data: doc } = await supabase
    .from("documents")
    .select(`
      *,
      order:orders(
        *,
        customer:customers(*),
        items:order_items(*)
      )
    `)
    .eq("id", params.id)
    .single();

  if (!doc) return NextResponse.json({ error: "单证不存在" }, { status: 404 });

  // 2. 查询公司信息
```

```
const { data: settings } = await supabase
  .from("system_settings")
  .select("key, value")
  .in("key", ["company_name", "company_name_en", "address", "email", "phone"]);

const companyInfo: any = {};
settings?.forEach(s => companyInfo[s.key] = s.value);

// 3. 生成 PDF
const pdf = new jsPDF();

// 标题
pdf.setFontSize(20);
pdf.setFont("helvetica", "bold");
pdf.text("COMMERCIAL INVOICE", 105, 20, { align: "center" });

// 公司信息
pdf.setFontSize(10);
pdf.setFont("helvetica", "normal");
pdf.text(companyInfo.company_name || "RCA Auto Parts", 14, 35);
pdf.text(companyInfo.address || "", 14, 40);
pdf.text(Email: ${companyInfo.email || ""} Tel: ${companyInfo.phone || ""}, 14, 45);

// 发票信息
pdf.text(Invoice No: ${doc.code}, 140, 35);
pdf.text(Date: ${new Date().toLocaleDateString()}, 140, 40);

// 买方信息
pdf.text("To:", 14, 60);
pdf.text(doc.order?.customer?.name || "", 14, 65);
pdf.text(doc.order?.customer?.address || "", 14, 70);
pdf.text(doc.order?.customer?.country || "", 14, 75);

// 明细表
autoTable(pdf, {
  startY: 85,
  head: [
    ["No.", "Description", "OEM", "Qty", "Unit Price", "Amount"]
  ],
  body: doc.order?.items?.map((item: any, idx: number) => [
    idx + 1,
```

```

item.product_name || "",
item.oem_number || "",
`${item.quantity} ${item.unit} || "pcs"`,
`${item.unit_price}`,
`${(item.quantity * item.unit_price).toFixed(2)}`,
) || [],
theme: "grid",
headStyles: { fillColor: [54, 78, 91] },
});

// 总金额
const finalY = (pdf as any).lastAutoTable.finalY || 85;
pdf.setFontSize(12);
pdf.setFont("helvetica", "bold");
pdf.text(
  `Total: ${doc.order?.currency} || "USD" ${doc.order?.total_amount}`,
  196, finalY + 10, { align: "right" }
);

// 签字栏
pdf.setFontSize(10);
pdf.setFont("helvetica", "normal");
pdf.text("Signature: _____", 14, finalY + 40);
pdf.text("Terms: FOB Payment: T/T", 14, finalY + 50);

// 4. 上传 PDF 到 Storage
const pdfBuffer = pdf.output("arraybuffer");
const fileName = `${params.id}/commercial-invoice-${Date.now()}.pdf`;
const adminClient = createAdminClient();
const { error: uploadError } = await adminClient.storage
  .from("documents")
  .upload(fileName, pdfBuffer, { contentType: "application/pdf" });
if (uploadError) {
  return NextResponse.json({ error: uploadError.message }, { status: 500 });
}

```

```
const { data: urlData } = adminClient.storage
  .from("documents")
  .getPublicUrl(fileName);

// 5. 更新单证记录
await supabase
  .from("documents")
  .update({
    file_url: urlData.publicUrl,
    issued_date: new Date().toISOString().split("T")[0],
    status: "issued",
    updated_at: new Date().toISOString()
  })
  .eq("id", params.id);

return NextResponse.json({
  url: urlData.publicUrl,
  message: "PDF 生成成功"
});
}
```

请生成所有文件的完整代码。

第十六章 操作日志与系统设置

16.1 完整模块生成 Prompt

按 **Ctrl+I** 唤起 Builder，输入：

生成操作日志和系统设置两个模块。请创建以下文件：

1. src/app/api/audit-logs/route.ts (GET - 列表)
2. src/app/api/audit-logs/[id]/route.ts (GET - 详情)
3. src/app/(dashboard)/audit-logs/page.tsx
4. src/app/(dashboard)/audit-logs/[id]/page.tsx
5. src/components/audit-logs/field-diff.tsx (字段级 diff 展示)
6. src/app/api/system-settings/route.ts (GET, PUT)
7. src/app/api/exchange-rates/route.ts (GET, POST)
8. src/app/api/exchange-rates/sync/route.ts (POST - 同步外部 API)
9. src/app/(dashboard)/settings/page.tsx (3 Tab)
10. src/components/settings/company-info-form.tsx
11. src/components/settings/exchange-rate-manager.tsx
12. src/components/settings/business-rules-form.tsx

【字段 diff 组件完整代码】

```
"use client";

import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow } from
"@/components/ui/table";

interface FieldDiffProps {
  before: Record<string, any> | null;
  after: Record<string, any> | null;
  fieldLabels?: Record<string, string>;
}

export function FieldDiff({ before, after, fieldLabels = {} }: FieldDiffProps) {
  const allKeys = Array.from(new Set([
    ...(before ? Object.keys(before) : []),
    ...(after ? Object.keys(after) : []),
  ]));

  return (
    <Table>
      <TableHeader>
        <TableRow>
          <TableHead>字段</TableHead>
          <TableHead>旧值</TableHead>
          <TableHead>新值</TableHead>
        </TableRow>
      </TableHeader>
      <TableBody>
        {allKeys.map(key => {
```

```

const beforeVal = before?.[key];
const afterVal = after?.[key];
const changed = JSON.stringify(beforeVal) !== JSON.stringify(afterVal);
const isDeleted = beforeVal !== undefined && afterVal === undefined;
const isAdded = beforeVal === undefined && afterVal !== undefined;

return (
  <TableRow key={key}>
    <TableCell className="font-medium">
      {fieldLabels[key] || key}
    </TableCell>
    <TableCell className={isDeleted ? "text-red-500 line-through" : ""}>
      {beforeVal === null || beforeVal === undefined ? "-" : JSON.stringify(beforeVal)}
    </TableCell>
    <TableCell className={
      isAdded ? "text-green-600 font-bold" :
      changed ? "text-yellow-700 font-bold" : ""
    }>
      {afterVal === null || afterVal === undefined ? "-" : JSON.stringify(afterVal)}
    </TableCell>
  </TableRow>
);
}}
</TableBody>
</Table>
);
}

```

【系统设置 PUT API 完整代码】

```

import { NextRequest, NextResponse } from "next/server";
import { createClient } from "@lib/supabase/server";
import { logAction } from "@lib/audit";

export async function GET() {
  const supabase = createClient();

```

```
const { data, error } = await supabase
  .from("system_settings")
  .select("key, value");

if (error) return NextResponse.json({ error: error.message }, { status: 500 });

// 转为 key-value 字典
const settings: Record<string, any> = {};
data?.forEach(s => { settings[s.key] = s.value; });

return NextResponse.json({ data: settings });
}

export async function PUT(request: NextRequest) {
  const body = await request.json();
  const supabase = createClient();

  for (const [key, value] of Object.entries(body)) {
    const { error } = await supabase
      .from("system_settings")
      .upsert({
        key,
        value,
        updated_at: new Date().toISOString()
      });

    if (error) return NextResponse.json({ error: error.message }, { status: 500 });
  }

  await logAction({
    user_id: "system",
    module: "system_settings",
    action: "update",
    after_data: body,
    description: "更新系统设置",
  });

  return NextResponse.json({ message: "保存成功" });
}
```


请生成所有文件的完整代码。

第十七章 工作台 Dashboard

工作台是用户每天打开的第一屏，承担“一眼看清业务全局”的职责。

17.1 完整模块生成 Prompt

按 **Ctrl+I** 唤起 Builder，输入：

生成完整的工作台 Dashboard。请创建以下文件：

1. src/app/api/dashboard/route.ts (GET - 聚合所有 KPI + 待办 + 图表数据)
2. src/components/dashboard/kpi-card.tsx
3. src/components/dashboard/todo-list.tsx
4. src/components/dashboard/order-trend-chart.tsx (Recharts 双轴图)
5. src/components/dashboard/customer-distribution-chart.tsx (Recharts 饼图)
6. src/app/(dashboard)/page.tsx (组装所有组件)

【Dashboard API 完整代码】

```
import { NextResponse } from "next/server";

import { createClient } from "@lib/supabase/server";

export async function GET() {
  const supabase = createClient();

  // 并行查询所有数据
  const [
    customersRes, inquiriesRes, quotationsRes, ordersRes,
    purchaseReqRes, purchaseOrdersRes, shipmentsRes, documentsRes,
    overdueRecRes, overduePayRes, pendingOrdersRes, pendingPrRes,
    expiringQuotationsRes, orderTrendRes, customerDistRes
  ] = await Promise.all([
    supabase.from("customers").select("*", { count: "exact", head: true }).is("deleted_at", null),
    supabase.from("inquiries").select("*", { count: "exact", head: true }).is("deleted_at", null),
    supabase.from("quotations").select("*", { count: "exact", head: true }).is("deleted_at", null),
    supabase.from("orders").select("*", { count: "exact", head: true }).is("deleted_at", null),
    supabase.from("purchase_requests").select("*", { count: "exact", head: true }).eq("status", "pending"),
    supabase.from("purchase_orders").select("*", { count: "exact", head: true }).is("deleted_at", null),
```

```
supabase.from("shipments").select("*", { count: "exact", head: true }).is("deleted_at", null),
supabase.from("documents").select("*", { count: "exact", head: true }).is("deleted_at", null),
supabase.from("receivables").select("amount", { count: "exact" }).eq("status", "overdue"),
supabase.from("payables").select("amount", { count: "exact" }).eq("status", "overdue"),
supabase.from("orders").select("*", { count: "exact", head: true }).eq("status", "pending"),
supabase.from("purchase_requests").select("*", { count: "exact", head: true }).eq("status",
"pending"),

// 7天内到期的报价
supabase.from("quotations").select("*", { count: "exact", head: true })
.eq("status", "sent")
.gte("valid_until", new Date().toISOString().split("T")[0])
.lte("valid_until", new Date(Date.now() + 7 * 86400000).toISOString().split("T")[0]),

// 近6个月订单趋势
supabase.rpc("get_order_trend"),

// 客户国家分布
supabase.from("customers").select("country").is("deleted_at", null),
]);

// 计算逾期应收总额
const overdueRecAmount = overdueRecRes.data?.reduce((sum: number, r: any) =>
sum + Number(r.amount), 0) || 0;
const overduePayAmount = overduePayRes.data?.reduce((sum: number, p: any) =>
sum + Number(p.amount), 0) || 0;

// 客户国家分布聚合
const countryCount: Record<string, number> = {};
customerDistRes.data?.forEach((c: any) => {
countryCount[c.country] = (countryCount[c.country] || 0) + 1;
});
const customerDistribution = Object.entries(countryCount)
.map(([country, count]) => ({ country, count }))
.sort((a, b) => b.count - a.count)
.slice(0, 8);

return NextResponse.json({
```

```

kpis: {
  customers_count: customersRes.count ?? 0,
  inquiries_count: inquiriesRes.count ?? 0,
  quotations_count: quotationsRes.count ?? 0,
  orders_count: ordersRes.count ?? 0,
  pending_approvals: 0, // TODO
  purchase_requests_count: purchaseReqRes.count ?? 0,
  purchase_orders_count: purchaseOrdersRes.count ?? 0,
  arrivals_count: 0, // TODO
  inbound_count: 0, // TODO
  shipments_count: shipmentsRes.count ?? 0,
  outbound_count: 0, // TODO
  documents_count: documentsRes.count ?? 0,
},
todos: {
  overdue_receivables: { count: overdueRecRes.count ?? 0, amount: overdueRecAmount },
  overdue_payables: { count: overduePayRes.count ?? 0, amount: overduePayAmount },
  pending_orders: pendingOrdersRes.count ?? 0,
  pending_purchase_requests: pendingPrRes.count ?? 0,
  expiring_quotations: expiringQuotationsRes.count ?? 0,
},
charts: {
  order_trend: orderTrendRes.data ?? [],
  customer_distribution: customerDistribution,
},
});
}

```

【KPI 卡片组件完整代码】

```

"use client";

import Link from "next/link";
import { LucideIcon } from "lucide-react";
import { cn } from "@/lib/utls";

```

```

interface KpiCardProps {
label: string;
value: number | string;
icon: LucideIcon;
color: string;
link: string;
}

export function KpiCard({ label, value, icon: Icon, color, link }: KpiCardProps) {
return (
<Link
href={link}
className="bg-white rounded-lg p-4 hover:shadow-md transition border border-gray-200"
>
<div className="flex items-center justify-between mb-2">
<span className="text-sm text-gray-500">{label}</span>
<Icon className={cn("h-4 w-4", color)} />
</div>
<div className="text-2xl font-bold">{value}</div>
</Link>
);
}

```

【订单趋势图 - Recharts 双轴图完整代码】

```

"use client";

import { LineChart, Line, BarChart, Bar, XAxis, YAxis, CartesianGrid,
Tooltip, ResponsiveContainer, ComposedChart } from "recharts";

interface OrderTrendChartProps {
data: { month: string; count: number; amount: number }[];
}

export function OrderTrendChart({ data }: OrderTrendChartProps) {
return (
<ResponsiveContainer width="100%" height={300}>

```

```

<ComposedChart data={data}>
  <CartesianGrid strokeDasharray="3 3" stroke="#f0f0f0" />
  <XAxis dataKey="month" stroke="#888" fontSize={12} />
  <YAxis yAxisId="left" stroke="#888" fontSize={12} />
  <YAxis yAxisId="right" orientation="right" stroke="#888" fontSize={12} />
  <Tooltip
    contentStyle={{ background: "fff", border: "1px solid #e0e0e0", borderRadius: 8 }}
  />
  <Bar yAxisId="right" dataKey="amount" fill="#3298cb" name="金额" />
  <Line
    yAxisId="left"
    type="monotone"
    dataKey="count"
    stroke="#c76747"
    strokeWidth={2}
    name="订单数"
  />
</ComposedChart>
</ResponsiveContainer>
);
}

```

【客户分布饼图完整代码】

```

"use client";

import { PieChart, Pie, Cell, ResponsiveContainer, Legend, Tooltip } from "recharts";

const COLORS = ["#3298cb", "#c76747", "#437453", "#9c7d3d", "#567a9e", "#94524c", "#417f9e", "#8a6b5a"];

interface CustomerDistributionChartProps {
  data: { country: string; count: number }[];
}

export function CustomerDistributionChart({ data }: CustomerDistributionChartProps) {
  return (
    <ResponsiveContainer width="100%" height={300}>

```

```

<PieChart>
<Pie
data={data}
dataKey="count"
nameKey="country"
cx="50%"
cy="50%"
outerRadius={80}
label={({entry: any}) => `${entry.country}: ${entry.count}`}
>
{data.map((_, index) => (
<Cell key={index} fill={COLORS[index % COLORS.length]} />
))}
</Pie>
<Tooltip />
<Legend />
</PieChart>
</ResponsiveContainer>
);
}

```

【工作台主页完整代码】

```

"use client";
import { useQuery } from "@tanstack/react-query";
import { Users, MessageSquare, FileText, ShoppingCart,
ClipboardList, Truck, Ship, DollarSign, FolderOpen,
History, Package, Warehouse } from "lucide-react";
import { KpiCard } from "@/components/dashboard/kpi-card";
import { OrderTrendChart } from "@/components/dashboard/order-trend-chart";
import {
  CustomerDistributionChart
} from "@/components/dashboard/customer-distribution-chart";
import { Badge } from "@/components/ui/badge";
import Link from "next/link";

```

```
export default function DashboardPage() {
  const { data, isLoading } = useQuery({
    queryKey: ["dashboard"],
    queryFn: async () => {
      const res = await fetch("/api/dashboard");
      return res.json();
    },
  });

  if (isLoading) return <div>加载中...</div>;

  return (
    <div className="space-y-6">
      <h1 className="text-2xl font-bold">早上好， 管理员</h1>

      {/ KPI 卡片 /}

      <div className="grid grid-cols-4 gap-4">

        <KpiCard label="客户数" value={data.kpis.customers_count} icon={Users} color="text-blue-500"
          link="/customers" />

        <KpiCard label="询盘数" value={data.kpis.inquiries_count} icon={MessageSquare}
          color="text-green-500" link="/inquiries" />

        <KpiCard label="报价数" value={data.kpis.quotations_count} icon={FileText}
          color="text-purple-500" link="/quotations" />

        <KpiCard label="订单数" value={data.kpis.orders_count} icon={ShoppingCart}
          color="text-orange-500" link="/orders" />

        <KpiCard label="请购数" value={data.kpis.purchase_requests_count} icon={ClipboardList}
          color="text-cyan-500" link="/purchases" />

        <KpiCard label="采购数" value={data.kpis.purchase_orders_count} icon={Truck}
          color="text-pink-500" link="/purchases" />

        <KpiCard label="发货数" value={data.kpis.shipments_count} icon={Ship} color="text-indigo-500"
          link="/shipments" />

        <KpiCard label="单证数" value={data.kpis.documents_count} icon={FolderOpen}
          color="text-teal-500" link="/documents" />

      </div>

      {/ 待办提醒 /}

      <div className="bg-white rounded-lg p-4 border border-gray-200">

        <h3 className="font-bold mb-3">待办提醒</h3>
```

```
<div className="space-y-2">
  <Link href="/finance?status=overdue" className="flex items-center justify-between p-2
  hover:bg-gray-50 rounded">
    <span className="text-sm">逾期应收</span>
    <Badge variant="destructive">{data.todos.overdue_receivables.count}</Badge>
  </Link>
  <Link href="/finance?status=overdue" className="flex items-center justify-between p-2
  hover:bg-gray-50 rounded">
    <span className="text-sm">逾期应付</span>
    <Badge variant="destructive">{data.todos.overdue_payables.count}</Badge>
  </Link>
  <Link href="/orders?status=pending" className="flex items-center justify-between p-2
  hover:bg-gray-50 rounded">
    <span className="text-sm">待处理订单</span>
    <Badge variant="secondary">{data.todos.pending_orders}</Badge>
  </Link>
  <Link href="/purchases?status=pending" className="flex items-center justify-between p-2
  hover:bg-gray-50 rounded">
    <span className="text-sm">待处理请购</span>
    <Badge variant="secondary">{data.todos.pending_purchase_requests}</Badge>
  </Link>
  <Link href="/quotations?status=expiring" className="flex items-center justify-between p-2
  hover:bg-gray-50 rounded">
    <span className="text-sm">即将过期报价</span>
    <Badge variant="outline">{data.todos.expiring_quotations}</Badge>
  </Link>
</div>
</div>

{/ 图表 /}

<div className="grid grid-cols-2 gap-4">
  <div className="bg-white rounded-lg p-4 border border-gray-200">
    <h3 className="font-bold mb-3">近 6 个月订单趋势</h3>
    <OrderTrendChart data={data.charts.order_trend} />
  </div>
```



```
<div className="bg-white rounded-lg p-4 border border-gray-200">
<h3 className="font-bold mb-3">客户国家分布</h3>
<CustomerDistributionChart data={data.charts.customer_distribution} />
</div>
</div>
</div>
);
}
```

请生成所有文件的完整代码。

第十八章 部署上线与持续迭代

18.1 Vercel 部署方案（推荐）

Vercel 是 Next.js 官方推荐的部署平台，免费版足够小型 ERP 使用。

步骤 1：代码推送到 GitHub

在 Trae 终端执行：

```
git init
git add .
git commit -m "Initial commit: RCA6.0 ERP"
git branch -M main
git remote add origin https://github.com/你的用户名/rca-erp.git
git push -u origin main
```

步骤 2：在 Vercel 导入项目

访问 <https://vercel.com> 用 GitHub 登录。点击 "Add New Project" → 选择 rca-erp 仓库 → 点击 "Import"。

步骤 3：配置环境变量

在 Vercel 项目设置页 → Environment Variables，添加与 .env.local 相同的变量：

- NEXT_PUBLIC_SUPABASE_URL
- NEXT_PUBLIC_SUPABASE_ANON_KEY
- SUPABASE_SERVICE_ROLE_KEY
- NEXT_PUBLIC_APP_NAME=RCA6.0 ERP
- NEXT_PUBLIC_APP_URL=https://你的项目名.vercel.app

步骤 4：部署

点击"Deploy"按钮，Vercel 自动执行 `pnpm build` 并部署。约 2-3 分钟后完成，得到一个 `https://xxx.vercel.app` 的访问地址。

步骤 5：配置自定义域名（可选）

在 Vercel 项目设置 → Domains，添加自己的域名（如 `erp.yourcompany.com`）。按提示在域名注册商处添加 CNAME 记录。Vercel 自动配置 HTTPS。

18.2 阿里云 ECS 自部署方案

如果公司要求数据私有化或 Vercel 访问不稳定，可以用阿里云 ECS 自部署。

步骤 1：购买 ECS 实例

在阿里云控制台购买 ECS：2 核 4G 内存，40G 系统盘，Ubuntu 22.04 系统，按量付费或包月。建议选杭州或北京区域。

步骤 2：安装 Docker

SSH 连接服务器后执行：

```
curl -fsSL https://get.docker.com | sh
systemctl enable docker
systemctl start docker
apt install -y docker-compose
```

步骤 3：创建 Dockerfile

按 `Ctrl+I` 唤起 Builder，输入：

在项目根目录创建 Dockerfile，多阶段构建：

```
# Stage 1: 安装依赖
FROM node:20-alpine AS deps
WORKDIR /app
COPY package.json pnpm-lock.yaml ./
RUN npm install -g pnpm && pnpm install --frozen-lockfile

# Stage 2: 构建
FROM node:20-alpine AS builder
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN pnpm build

# Stage 3: 运行
FROM node:20-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production
COPY --from=builder /app/public ./public
COPY --from=builder /app/.next/standalone ./
COPY --from=builder /app/.next/static ./next/static
EXPOSE 3000
CMD ["node", "server.js"]
```

注意：需要在 `next.config.mjs` 中添加 `output: "standalone"` 配置。

同时创建 `docker-compose.yml`，包含 `erp` 服务和 `nginx` 反向代理服务，把 80 端口转发到 3000。

步骤 4：部署到服务器

```
git clone 代码到服务器
docker-compose up -d --build
```

访问 `http://服务器IP` 验证部署成功。

步骤 5: 配置 HTTPS

```
apt install certbot python3-certbot-nginx
certbot --nginx -d erp.yourcompany.com
```

证书 90 天自动续期。

18.3 性能优化

优化项	方法	收益
数据库索引	常用筛选字段建索引	查询快 5-10 倍
API 缓存	工作台数据缓存 5 分钟	减少 80% DB 查询
前端 Bundle	dynamic import 拆分大组件	首屏快 30-50%
图片优化	next/image 自动优化	体积减 60%
CDN 加速	Vercel CDN 或 Cloudflare	全国访问加速

18.4 监控与告警

推荐用 Sentry 监控错误（免费版 5000 错误/月）：

```
pnpm add @sentry/nextjs
npx @sentry/wizard@latest -i nextjs
```

18.5 后续迭代 workflow

上线后的迭代遵循"三步法"：

- 1. 描述变更：一句话说清楚要改什么
- 2. 让 Trae 改代码：把变更描述+相关现有代码+数据库 schema 喂给 Trae Builder
- 3. 测试与合并：运行测试用例，PR review 后合并

Git 工作流：

```
git checkout -b feature/xxx
# Trae Builder 修改代码
pnpm test # 运行测试
git add . && git commit -m "feat: 描述"
git push origin feature/xxx
# GitHub 创建 PR → Code Review → 合并 main → Vercel 自动部署
```

教程总结：完成本教程后你已经掌握了：① 用 Trae 从零搭建 Next.js 全栈项目 ② 设计和实现 14 模块 ERP 系统 ③ 实现订单状态机和事务性副作用 ④ 部署到 Vercel 或阿里云 ⑤ 持续迭代方法论。AI 是放大器——配合良好的工程纪律，你能做到传统团队 3-5 倍的产出。

附录 A：Trae 快捷键速查

A.1 AI 相关

快捷键	功能	使用场景
Ctrl+L	唤起 Chat 对话面板	问问题、生成代码片段
Ctrl+I	唤起 Builder 内联输入	多文件生成、批量修改
Tab	接受 AI 代码补全	编辑代码时自动弹出
Esc	拒绝补全	不想用 AI 建议时
Ctrl+Shift+L	选中文件作为上下文	让 AI 基于当前文件回答
Ctrl+Shift+Enter	解释选中代码	理解陌生代码

A.2 文件与编辑

快捷键	功能
Ctrl+P	快速打开文件
Ctrl+Shift+P	命令面板
Ctrl+S	保存
Ctrl+Z / Ctrl+Y	撤销 / 重做
Ctrl+X (未选中)	剪切当前行
Ctrl+C (未选中)	复制当前行
Ctrl+D	选中下一个相同词
Ctrl+Shift+K	删除当前行
Alt+ ↑ / ↓	上下移动当前行
Ctrl+/	切换行注释

A.3 导航与终端

快捷键	功能
Ctrl+G	跳转到行
Ctrl+`	打开/关闭终端
Ctrl+B	切换侧边栏
Ctrl+Shift+E	聚焦资源管理器
Ctrl+Shift+F	全局搜索
Ctrl+\	拆分编辑器
Ctrl+W	关闭当前标签

附录 B：常见报错与解决

E1: Module not found: @/components/ui/xxx

- 原因：shadcn 组件未安装

- 解决: `npx shadcn-ui@latest add xxx`

E2: Supabase connection refused

- 原因: 环境变量配置错误
- 解决: 检查 `.env.local`, 重启 `pnpm dev`

E3: RLS blocked the query

- 原因: Supabase RLS 未关闭
- 解决: `ALTER TABLE 表名 DISABLE ROW LEVEL SECURITY;`

E4: Hydration mismatch

- 原因: 服务端和客户端渲染不一致
- 解决: 把 `Date.now()/Math.random()` 放到 `useEffect`

E5: Cannot read properties of undefined

- 原因: 访问 `undefined` 对象的属性
- 解决: 用可选链 `data?.items?.[0]?.name`

E6: CORS error

- 原因: 前端跨域被拦截
- 解决: API 路径以 `/api` 开头, 同源访问

E7: 401 Unauthorized

- 原因: 未登录或 `session` 过期
- 解决: 检查 `middleware` 和 `Supabase Auth session`

E8: Unique constraint violation

- 原因: 唯一约束冲突
- 解决: 检查 `generateCode`, 或用 `ON CONFLICT DO NOTHING`

E9: 500 Internal Server Error

- 原因: 服务端异常
- 解决: 查看终端堆栈, 复制给 Trae Chat 修复

E10: TypeScript type mismatch

- 原因: 类型不匹配
- 解决: 重新生成 `supabase gen types`

E11: useSearchParams should be in suspense

- 原因: `useSearchParams` 未包 `Suspense`
- 解决: `<Suspense fallback={...}><Comp/></Suspense>`

E12: Hooks can only be called in client component

- 原因: `Server Component` 中用了 `Hook`
- 解决: 文件顶部加 `"use client"`

E13: pnpm: command not found

- 原因: `pnpm` 未安装

- 解决: `npm install -g pnpm`

E14: Port 3000 is already in use

- 原因: 3000 端口被占用
- 解决: `lsof -i :3000` 找进程并 kill, 或换端口

E15: Git failed to push some refs

- 原因: 远程有更新未拉取
- 解决: `git pull origin main` 解决冲突后重推

结语

完成本教程意味着你已经从一个零基础学习者, 成长为能够独立用 AI 工具开发企业级 ERP 系统的"AI 全栈工程师"。这不是终点, 而是新起点——AI 编程的能力边界还在快速扩张, 今天学到的 Trae 用法明年可能就被更强大的工具取代。但底层方法论（需求分析、数据库设计、状态机、事务、审计、测试、部署）是不会过时的。

建议接下来的三个月:

1. 把本教程的 ERP 系统真正用起来, 发现不足持续迭代
2. 学习一个新 AI 工具（如 Cursor 或 Claude Code）, 对比不同工具的优劣
3. 尝试一个新项目（如 CRM、OA、WMS）, 把这套方法论迁移过去

三个月后回头看, 你会发现自己已经完成了从"会用 AI"到"善用 AI"的蜕变。

祝你编程愉快!